28 OCTOBER 2025 • MANUAL

POT JS

MANUAL

POT JS is a library for Javascript that allows to effortlessly store, update and retrieve values to a distributed, always-on storage network.

Under the hood it is a Javascript API for the Go Proximity Order Tree (POT) implementation that allows to write and read key-value pairs to and from the decentralized, immutable storage network Swarm.

The POT storage tree structure allows to have updateable key-value maps stored on an otherwise immutable, decentralized, content-addressable network like Swarm. It is called Proximity Order Tree after a fundamental, new storage technique that is described in a forthcoming paper.

This manual covers the use of POT JS, as well as hints and caveats for maintainers and contributors to the project.

INDEX

USE

QUICK START

- Clone **http://github.com/brainiac-five/potjs**
- Drop **lib/pot.wasm**, and **pot-web.js** or **pot-node.js** in a directory.
- Write some Javascript using functions **new()**, **put()**, **get()**, **save()**, and **load()**.
- Check out the function overview (pg. 40), reference (separate doc) and examples (pg. 42)

There is no need to *build* the project, i.e., no need to install or run Go, to *use* it. Only **pot.wasm** and either **pot-web.js** or **pot-node.js** are needed, as they come. They are in the **lib/** folder.

EXAMPLE

```
<script src="pot-web.js"></script>

<script>

(async () => {

    await pot.ready()
    map = await pot.new()
    await map.put("hello", "POT!")
    value = await map.get("hello")
    console.log("key ‹hello› has:", value)
})()

</script>
```

To run this example:

```
cd potjs/tutorial
npx http-server -c1 . &
open http://localhost:8080/t0.html
```

Check the browser console.

tutorial/t0.html

Node.js works analogous, see **examples/example7.js**, pg. xxx

INTRODUCTION

This is an introduction to how POT JS is used. The basics are simple and fit on this one page.

You may then choose to work through the structured explanation that follows to get a deeper understanding; or skip to the hands-on tutorial (pg. 17). Or You might decide to check out the (separate) function reference after a cursory look at the following pages, to beat your own path. The final chapters of this manual explain the inner workings of POT JS to contributors and maintainers (from pg. 61).

BRIEF

POT JS provides a simple way to store and retrieve mutable key-value pairs on the immutable, decentralized storage network Swarm.[1]

The main primitives are:

```
kvs = await pot.new(swarm, batch)
await kvs.put("some key", "some value")
value = await kvs.get("some key")
```

This almost constitutes a complete example program, cf. example 1 (pg. 43).

The implementation caches the data in-memory. To write the entire key value store to a network, and retrieve data back from it, use:

```
ref = await kvs.save()
kvs = await pot.load(ref)
```

There are less-essential functions for raw byte access, for synchronous access, and for tests. (See the function overview, pg. 40.)

To use POT JS in the browser, include:

```
<script src="pot-web.js"></script>
```

For node.js, require:

```
require("pot-node")
```

[1] https://ethswarm.org/

PROXIMITY ORDER TRIE

POT JS makes Proximity Order Trie (POT) functionality available in the browser or with node.js. The following is a brief, high-level description of what it is for and how it works.

A proximity order tree is a data structure that allows to operate a key-value store on an immutable, content-addressed[2] storage network. It makes it possible to effortlessly use Swarm as a key-value database; or to store a map-like structure to it – including being able to change values.

This bags two oxymorons: an immutable storage would seem to not allow for any update of values nor adding of additional key-value pairs. And Swarm is content-addressed: a hash of the content (not an arbitrary key) constitutes the address of a value in the network.

POT solves this by exploiting a feature observed in Merkle tries when hashing its nodes: after an update, there is one path back to the root that has to be re-hashed to arrive at a new root hash. All other hashes in the trie remain the same, as no values that are part of their pre-images have changed. But the root hash always changes. That's the point with Merkle trees. POT uses this attribute to implement an update to data stored in the proximity order trie as a change of the root hash.[3]

Because a network like Swarm functionally provides the inverse to the hashing process – it returns the pre-image to the hash, by the very definition of content-addressing – the update process of a Merkle trie can be reversed and repurposed as the re-indexing mechanism of a trie that stores content-addressed data. A new root hash will be created on every update that serves as the new entry point to the respectively now-valid tree of data entries. Deleted nodes are not present anymore, because they fell out of the tree, even if they are still stored. Updated values are sorted into the tree with their new address. All entries, old and new, still exist in-storage. But they are not part of the hash tree anymore. The nodes that held the key-value pair of an updated pair that now has a new value are left out as their combined key and value contents has changed its hash.

The keys of this key value storage, though, are not native Swarm keys, which is made for retrieving pre-images of hashes. To search for a key to retrieve its arbitrary, not-pre-image value is therefore not something Swarm's base layer can do. POT uses specific bit patterns to implement navigation of this data store at a useful pace. That is, POT is the complex algorithm that makes the system 'understand' where to find the value to a key, as well as to understand where to insert a new value associated with a new key. Concretely, a proximity order trie uses the bit patterns of a key's hash to establish a logarithmic-time search order. It is described in a forthcoming paper.

POT JS uses the implementation of POT in Go that performs the actual number crunching for this:

https://github.com/ethersphere/proximity-order-trie

[2] Content-addressed means that the storage address is produced from the value that is stored. If the value changes, the address changes, too. A value can therefore not be updated without losing its address.

[3] The values of this KVS are not the values stored in Swarm. They have keys that are independent of their values. Key and value are stored together and constitute the value of the Swarm record.

CODING

POT JS is for storing key-value sets on Swarm. It works in the browser and with node.js.

This chapter explains the basics of coding with POT JS. It concludes with an explanation of concepts. Also see the examples (pg. 42), the tutorial (pg. 17), the function overview (pg. 40) and the function reference (separate document in **docs/**).

INITIALIZATION

BROWSER

For use in the browser, to initialize, include **pot-web.js**, then wait for the initialization to finish:

```
<script src="pot-web.js"></script>
<script>
(async () => {
   await pot.ready()
   ...
```

XXX

If **pot.wasm** is not in the same folder as **pot-web.js**, a path to **pot.wasm** can be provided using the **wasm** attribute of the script tag.

```
<script src="pot-web.js" wasm="lib/pot.wasm"></script>
```

XXX

NODE.JS

For use with node.js, include **pot-node.js**, then wait for the initialization to finish:

```
require("pot-node")
; (async () => {
    await pot.ready()
   ...
```

XXX

If a path to **pot.wasm** must be provided, add an additional parameter to **require** like this:

```
require("pot-node")("lib/pot.wasm")
```

XXX

DATA STORAGE

After `pot.ready()` resolves, the JS `pot` object can be used to create a new key value store (KVS) with `pot.new()`. Use its `put()` and `get()` methods to store and retrieve key-values pairs to and from it.

If the `new()` call has no arguments – like in the example below – **in-memory storage** is used for the KVS, which can be useful for testing and prototyping. In this mode, values are not permanently stored to a network but are lost when the program is stopped. `pot.new(null, null)` has the same effect.

The call to `pot.new()` determines where a KVS will be persisted, on which network, or in-memory. For networks, it receives two parameters: the URL of the API of a client of the network, and a postage batch id to pay for the storage. This is explained in more detail below (pg. 9).

```
kvs = await pot.new()
await kvs.put("hello", "Hello, P.O.T.!")
value = await kvs.get("hello")
```

XXX

See examples/`example1.html` (pg. xxx) for a minimal but complete program using these POT JS calls.

PERSISTING

Any `put()` is by default cached. To persist the KVS, use `save()`. To retrieve it at a later point, `load()`.

```
ref = await kvs.save()
kvs2 = await pot.load(ref)
```

The `save` persists the KVS – all its key-value pairs – to the storage designated by the `new` call's arguments (see below).

These operations only make sense when using a network as storage. The in-memory mode will lose all storage when a program stops. The `load` and `save` methods are available, too, though, when using in-memory storage to facilitate development and testing. It makes the tutorial and examples quick and easy to replicate.

THE SAVE REFERENCE

The reference returned from `kvs.save()` is the root of a POT trie. It identifies one specific state of the set of key-values that is a KVS.

This reference is not tied to the identity of a specific KVS object, but, like a hash belongs to its pre-image, represents a specific state. It is not tied to a specific variable, or Javascript, and the reference is, of course, not tracking the changes to the KVS after the save. It will at all times return the same KVS state through `load()` – i.e., the same set of key-value pairs – until the postage batch (see below) expires and the data is purged from Swarm; or until a Swarm test network that was used for storing it is shut down.

A different reference will be returned upon saving the same KVS object when it was updated in the meantime (values added or changed). The same reference will retrieve the same KVS, regardless of whether the KVS that was originally saved is later updated and saved again or not, or a KVS that was loaded through this reference gets updated, and saved again or not. Any updated KVSs would have a different reference returned from `save()` after any update; unless the updates result into the same state as when it was saved. And creating the exact same KVS by the same sequence of `put()` calls will result in the same reference when saving.

This is true for in-memory storage the same as when storing to a network.

KEY AND VALUE TYPES AND SIZE LIMIT

A key can be a string, a number, or a Uint8Array of bytes. It can be up to 32 bytes.

A value can be a string, a number, a Uint8Array of bytes, or a Boolean.

The limit for value size depends on the system set up. It is currently markedly narrower for node.js: It can be at minimum 100,000 bytes for node.js, a 1,000,000 bytes in the browser. This limitation, especially for node.js, is likely to be improved in the future.

A soft cap for value size exists that defaults to 100,000 byte. Trying to store a value bigger than this size will result into an error. The cap can be arbitrarily set to a different value using `setValueLimit()`. It does not per se change what the underlying system will be able to absorb.

VALUE TYPES

Using `put()` and `get()` (or `putSync()` or `getSync()`), the type that was stored is returned.

But this is not the only way. Values can be stored with `putRaw()` (or `putRawSync()`), in which case no information is stored with the data regarding what type it was. There are *raw* access functions that deal with types on *getting* raw data: `getBoolean()`, `getNumber()`, `getString()`. Their function names basically indicate what type a raw value will be type cast to. They are for reading data stored with `putRaw()`, where only one byte is stored for Booleans, 8 for numbers, etc. without the type information. This may be useful in certain situations but is incompatible with the normal `put()` and `get()` functions. Data stored with `put()` has a type byte prepended that will show up when reading it back with `getRaw()`. Data stored with `putRaw()` does not have that leading type byte and reading it back with `get()` will most of the time lead to wrongly interpreted value or error messages.

These competing approaches overlap with a simple third one that just stores and retrieves raw `Uint8Array`s, using only `putRaw()`, and `getRaw()`. This leaves the task to the retrieving code to cast into the right value.

KEYS

Keys can be Javascript `numbers`, up to 32 byte long `strings`, or `Uint8Arrays`.

Keys are not bijective: there are multiple ways to represent the same byte sequence that a key consists of and there is no type-coding for keys as there is for values:

Numbers are converted to their float byte representation, so the key becomes a 8-byte value, followed by 24 zero bytes. There are no Big Int keys. All keys are internally padded with zeros for a size of 32 bytes. Keep present that Unicode characters may use more than one byte per character. `s.length` returns the number of bytes used in a string `s`.[4]

Thus, `"1"`, `1`, and `"0x01"` are different keys but `1`, `1.0`, `0x1` and `01` are the same. Precision of decimal fractions or very high numbers – e.g., when using literals – can be lost the same way it is lost for Javascript's floats. The diverse `NaN` error conditions[5] are effectively different keys. This conversion, instead of going by digits, makes sense because there is no strong way in Javascript to tell integers and floats apart as Javascript knows only one unified Number type that is 64 bit IEEE 754. Strings

[4] https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/String/length

[5] There are many ways to have an invalid bit pattern or IEEE 754, and different codes for "not a number" (`NaN`), depending on how it happened. POT JS simply takes what is in the float variable and uses the float format bit pattern. While they are represented the same ("NaN"), different `NaN` codes might not be the same bit pattern.

and byte arrays are length-checked to be <= 32 byte and cause an error if the limit is violated. The byte arrays `[0]`, `[0,0]` etc. are the same key, as is `[1]`, `[1,0]`, or `[1, 0, 0]` because of the internal zero padding. The byte array `[65]` is the same key as `131,072` (because `0x4100000000000000` represents `131,072` in 64 bit IEEE 754) , `"A"`, or `"A\0"` (`because 0x41 is UTF-16 for "A"`) , so is `[65, 0, 0, 0, 0, 0]`. They are all converted to the binary key

`0x4100000000000000000000000000000000000000000000000000000000000000`.

NETWORK CONNECTION

To connect with a Swarm network, the call to `pot.new()` requires two parameters:

1. a URL to a Swarm bee node http API and
2. a Swarm batch ID to pay for storage.[6]

For example (cf. `example5.html`), in the example below, the arguments to `new()` connect the KVS that is created with a local Swarm test network:

```
map = await pot.new("http://localhost:1633",
        "ab642e87327ec31816ed7aa675b4e92ea2c3d92c5022a599604c8bd8ddfe545d")
```

XXX

Note that the batch id – the second argument – has to be freshly created for your tests. See the next subchapter on how.

Another example, for node.js (cf. `example8.js`):

```
bee = process.argv[2]
batch = process.argv[3]
kvs = await pot.new(bee, batch)
```

XXX

An example invocation of this script snippet would be:

[6] For an intro to Swarm and the concept of its network nodes, the Bees, and the postage batch IDs that power its incentive mechanisms, see https://ethswarm.org. You do not need to operate a Swarm Bee node to use POT JS: you can try it locally, with in-memory storage during development and testing. It can be operated with a local Swarm test network. But to write data to the decentralized Swarm mainnet, you will need a postage batch.

```
% node hello.js http://localhost:1633 \
 6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa
```

Also here, note that the batch id – the third argument – has to be freshly created for your tests, otherwise the `put()` or `save()` calls will fail.[7]

BATCH ID

All tests and most examples can be tried out **without** a batch ID: the **in-memory** storage mode works without it, and are sufficient for learning, testing, and developing. But for production use you will likely want to store to Swarm mainnet.

A batch ID stands for **postage stamps** that Swarm uses for incentivization, inter alia, to compensate its Bee nodes to keep copies of the stored data. Batch IDs are the prime payment mechanism that Swarm offers, pervading most every aspect of decentralized data hosting on Swarm.

For trying out POT JS, it is **not** required to understand the concept of batch IDs, nor even for coding POT JS apps on a local Swarm testnet: there are `make` rules provided that set the local Swarm testnet up in a docker container and obtain a free batch id for your tests from z

But if you are planning to work with Swarm, it will be indispensable to understand the concept of batch IDs, how to buy them and for how much data, and for how long they are good:

https://docs.ethswarm.org/docs/develop/tools-and-features/buy-a-stamp-batch

INSTALLING A LOCAL SWARM NETWORK

The installation and firing up of a local test network is highly automated, you basically only need docker installed to test it with a single command, e.g.:

```
% make node_locnet_test
```

[7] Currently, with a misleading 404 error.

Batch IDs are required when working with a network instead of the in-memory mode. This make rule creates a free postage batch ID for the test and uses it.

All tests and most examples can be tried out **without** connection to any Swarm network: POT's in-memory mode allows for testing and developing without one.

But to test an app with a Swarm network, the easiest way is to use such make rule to install a local Swarm test network and run it. The make rules use this docker-based installation by fairdatasociety:

https://www.npmjs.com/package/@fairdatasociety/fdp-play

Alternatively, Swarm provides documentation on how to set up a local 2-node testnet:

https://docs.ethswarm.org/docs/develop/tools-and-features/starting-a-test-network/

The following script starts the Swarm test network, creates a new batch ID and stores it into the file **.batch_id**.[8] Using **make** to initiate tests and examples does that for you.

```
# starting five local swarm nodes
fdp-play start --detach

# buy stamps (free in this test setup)
swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b | tee .batch_creation
grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
echo "postage batch ID: $(cat .batch_id)"

# stopping nodes
fdp-play stop
```

cf. Makefile, rule node_locnet_test

The Makefile rule for example 9 (search for **"example9:"** in the file **Makefile**) is an example for how to start a local Swarm test network and create a postage batch for it on it. This batch is a free batch of stamps, created on your own local Swarm test network. It will only work there and only for 2 days.

EXPLICIT INITIALIZATION

For browser scripts (not for node.js), as an alternate way to initialize WASM, you may leave out **pot-web.js**: include only the generic Go WASM wrapper **wasm_exec.js**, from the **lib/** folder; create

[8] The file name starts with a dot and thus, by default, is not visible in Mac Finder, nor with **ls** in the terminal. Press **shift** + **command** + **.** or use **ls -a**.

an object called **Go** and instantiate the POT WASM executable **pot.wasm**, then run it once it is loaded. Cf. **examples/example3.html**.

```
const go = new Go()
WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
   .then((r)=>{
          go.run(r.instance)
          map = pot.newSync()
   })
...
```

Part of examples/example3.html

Then use `onPotInitialized()` for the start of your POT JS. Before that function is called, there will be no **pot** object.

```
async function onPotInitialized() {
   map = await pot.new()
   await map.put("hello", "POT!")
   value = await map.get("hello")
}
```

Part of examples/example3.html

This way to initialize **pot.wasm** may have benefits in some scenarios. For a more detailed discussion of it, see **Initialization**, pg. 78.

VERBOSITY

The verbosity log level controls how much output is generated by POT JS.

The default log level is **DEBUG**, printing a fair amount of details of processing steps and state change to the browser console or screen.

0	**pot.NONE**	no logging
1	**pot.CRITICAL**	logs only errors that would appear to arise from a malfunction of POT JS.
2	**pot.ERROR**	programming errors stemming from wrong use are also logged.
3	**pot.INFO**	general runtime information is logged.
4	**pot.DEBUG**	state changes, like binary level put and get keys and values are logged.
5	**pot.TRACE**	Some code is wired to report on every step of the way.

Normal operation will benefit from the INFO log level. DEBUG can help to find confirmation about assumptions while hunting programming errors also concerning the use of POT JS. TRACE makes sense for extending and optimizing POT JS itself. NONE, CRITICAL or ERROR remove most utterances from POT JS so that the focus can be elsewhere.

The verbosity can be adjusted with `pot.setVerbosity()`. The log prints to screen when running POT JS with node.js. In the browser, it logs into the browser console.

The default log level that a call to `pot.log()` uses is CRITICAL, which means that a call to `pot.log()` will show up unless verbosity of the system is set to NONE. A different log level can be passed to `pot.log()` as optional second parameter.

Cf. examples/example8.js:

```
(async () => {
    await pot.ready()
    pot.setVerbosity(pot.INFO)
    kvs = await pot.new()
    ...
})()
```

Part of examples/example8.js

If it is important to even suppress the first log line of POT JS, there are ways to set verbosity before the WASM executable is even initialized: setting `globalThis.potjs_verbosity;` adding the attribute `verbosity` to the script tag of `pot-web.js` in the browser; or setting a second parameter to require of `pot-node.js` in node.js. These latter ways to set verbosity all need a literal integer supplied, because the `pot.*` constants listed above do not exist yet at that point.

```
var go = new Go()
global.potjs_verbosity = 3
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
    .then((r) => { go.run(r.instance) })

onPotInitialized = async () => {
    ...
```

XXX

ERROR PROPAGATION

The error strategy of POT JS is Javascript-style to the degree possible, generally using JS exceptions triggered from the Go WASM functions via promise rejections, rather than returning error codes, Go-style. For the *Sync() functions that don't use promises though, an error is returned instead of thrown as there is no way of throwing a Javascript error directly from Go.

The implementation of this is in Go (**potjs.go**), not in Javascript, creating JS code in Go, for full data/situational access on the Go side (cf. pg. 73). A major limitation of the Go/Javascript interface **syscall/js** is that Javascript exceptions cannot be thrown from Go. What works is to call the reject parameter of in the executor of JS promises that are constructed in Go. Because the more relevant functions are promise-based at any rate, this covers the more common use cases. Synchronous functions, however, return JS error objects that have to be tested for an error where normally an exception would be expected.

CRASHES

There is a peculiar form of deadlocks and pseudo deadlocks emanating from POT JS's hybrid nature that a programmer should be aware of. If a deadlock happens, the WASM executable crashes and all state that it caches (that had not been persisted with **save()**) is lost. The easy way to minimize such problems is to avoid synchronous functions when working in the browser.

If the Go WASM executable, **pot.wasm**, deadlocks, or thinks it might be deadlocked and crashes it cannot be recovered. The volatile state of all KVS objects is lost, all POT JS functions stop working and throw an error instead when called that reports that the Go executable is down, and/or that a method of a KVS does not exist. In this case, **pot.wasm** must be re-initialized before the POT JS functions can be used again. This affects all **pot** and **kvs** functions, it means that while everything else might still work, the methods of a **kvs** that on the face look like native Javascript functions, are literally just 'gone.'

The reasons for such crashes are suspected or real deadlocks and other uncaught runtime errors – 'panics' – on the side of the Go executable. The capacity of deadlock errors is always there for a WASM architecture because of the fact that Javascript and Go WASM share into one thread. Since both runtimes do not have full transparency, the Go runtime deadlock detection can throw false positives.[9] Multiple measures are in place for POT JS to keep the WASM executable alive in all

[9] Javascript does not have a deadlock detection; it does not need one because it is not made for real multi-threading. Its asynchronicity is a programming paradigm that feels parallel but is implemented on one thread, which keeps the more involved challenges of parallel execution away from the programmer. Go is built to operate on multiple threads in parallel and one can therefore, relatively easily, create real deadlocks in Go with the means that exist to manage the parallelism. As Go is protective in nature, too, it comes with an automatic deadlock detection. But this protection can probably not operate well when it cannot 'see' where the main

scenarios, to handle panics gracefully and to not allow a deadlock to happen. To maximize robustness, one might still want to remain aware of this case, which means to avoid synchronous functions in the browser for production.

The capability to deadlock is not a fault of the Go POT implementation but lies in the single-thread nature of Javascript that can be programmed highly asynchronously but is not truly parallel, multi-threaded, or multi-process (except when using the duly limited Web Workers). That Javascript is designed this way provides strong safeguards for the programmer against a class of difficult to track errors. But the guardrails are coming off when crossing from Javascript into Go-land *and back*: a blocking wait in Go that hands control back to the browser or node.js deadlocks its main Javascript execution thread. Javascript waits for Go to continue and Go for Javascript. This is reliably the case when Go executes a network fetch that is internally handed back from WASM to Javascript to execute. When using synchronous POT JS methods to write to and read from Swarm, almost every function may use a fetch. When using one of the in-memory persistence models, none does.

FURTHER READING & SUPPORT

For more implementation details, check out the developer notes from page 73.

The tutorial follows, visiting a number of relevant angles to get a fuller picture.

Refer to the examples in the `examples/` folder and the Examples chapter (pg. 42) to understand how POT JS might be used in the specific use case you envision.

There is a useful function overview below (pg. 40). The (separate) function reference in the `doc/` folder explains all available functions with example snippets.

The source code of POT JS is hosted at `https://github.com/brainiac-five/potjs`.

The Go POT implementation is hosted at `github.com/ethersphere/proximity-order-trie`.

For more information on Swarm, visit `https://ethswarm.org`.

How to obtain postage batches is described at `https://www.ethswarm.org/get-bzz`.

Find support at `https://discord.ethswarm.org`.

thread is going, when control is really with the main Javascript thread, in Javascript land with nothing happening in Go. To the deadlock detection this might look as if things had stalled. This friction by necessity leaks into POT JS.

TUTORIAL

This is a step-by-step guide to learn to use POT JS.

In this tutorial, we will discuss the core building blocks, alternate options, and some special aspects that deserve mention. The code examples used and referred to in the captions are in the **tutorial/** folder. If you do as instructed, this folder will be at **./potjs/tutorial** from your working directory.

We will start out with a minimal node.js example, then do the same in the browser, develop it into a more interactive experience that is a primitive web app, then shift functionality back from the browser to the server-side to work with node.js again. Finally, we'll use Docker to run a local Swarm test network and check out how connection to a real storage backend works.

The matter is simple but it will help to get a bit of background, too, as is provided below. If you like it faster, at any point checking out the **examples/** folder may be all you need to get going. The examples are described from pg. 42; then use the **function reference** document in the **doc/** folder.

INSTALLATION

All of the following works with node.js 22.17.1 on a Mac.

Create a new directory, and clone POT JS into it from github:

```
mkdir tut
cd tut
git clone https://github.com/brainiac-five/potjs
```

There is no building or installation required beyond this.

The files we need are

- **lib/pot.wasm**
- **lib/pot-web.js**
- **lib/pot-node.js**

These are the Go implementation of POT with JS interface compiled to WASM – **pot.wasm** – and the Javascript initialization code for it, for in-browser and node.js respectively. The files are portable across operating systems and browsers and run on Mac and Ubuntu.

HELLO, P.O.T.!

The main functions of POT JS are **put()** and **get()**. Maps are created with **new()**. They are saved to the network (or to an in-memory trie) by **save()** and retrieved with **load()**.

For starters, we will work with **new()**, **put()** and **get()**.

For our first file, In **tut/** – the folder you created for this tutorial and cloned potjs into –, create a file **t1.js** and paste this:

```
require("./potjs/lib/pot-node")

; (async () => {
      await pot.ready()
      kvs = await pot.new()
      await kvs.put("hello", "P.O.T.")
      value = await kvs.get("hello")
      console.log("hello:", value)
}
```

tutorial/t1.js

You also find this script in **potjs/tutorial/t1.js**.

The first line requires the Javascript wrapper around **lib/pot.wasm** and initializes it. **pot.wasm** is a Go program compiled to WASM. Its main source file is **./potjs.go**. **pot.wasm** is loaded, started and keeps running. It creates Javascript objects that are written in Go. Most importantly at this point, it fleshes out **pot**.

```
require("./potjs/lib/pot-node")
```

In the odd case that **pot.wasm** was not in the same folder as **pot-node.js**, an additional argument to require can be used. We won't go through the moves on that but note that the extra parameter is not placed within the brackets but gets an extra set of brackets (see pg. 6).

The fourth line blocks until the initialization of **pot.wasm** – and with that, POT JS – is complete. That is, **pot.wasm** is loaded and its main function is done with creating the **pot** object.

```
await pot.ready()
```

The loading and starting of **pot.wasm** is an asynchronous affair.[10] And before the relevant **pot** calls can happen, the initialization needs to be complete. That's what **pot.ready()** waits for.

The next line creates a key-value store (KVS). **await pot.new()** is an asynchronous call but really nothing much happens at this point yet:

```
kvs = await pot.new()
```

Because we call **new()** without any parameters, the KVS will be stored in memory, not to a network. We will stick with this easier mode for exploring for a moment.

Next, we store a value to the KVS, **P.O.T.**, under the key **hello**:

```
await kvs.put("hello", "P.O.T.")
```

The way **pot.wasm** works, this value is cached and not persisted yet. Even if we were initializing the KVS to be stored to a network, at this point nothing would happen, yet.

Next, we'll read it back out from the KVS:

```
value = await kvs.get("hello")
```

And after that the value, **P.O.T.**, is logged to the console.

RUNNING IT

To run this script, do:

```
node t1.js
```

WHAT YOU SHOULD SEE

```
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
```

[10] The pot object is created early on in pot-node.js. But **pot.wasm** assigns its methods to it.

```
pot:  › slot ref: 1
pot:  » put  hello: P.O.T.
pot:  › ⟶  68656c6c6f000000000000000000000…: 03502e4f2e542e
pot:  » get  hello: P.O.T.
pot:  › ⟵  68656c6c6f000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

The default log level setting of POT JS is **DEBUG** level, so POT JS shares some internals.

In line 5 of the log, POT JS tells us that it is 'persisting' the key-value store in memory.

```
pot:  › created new in-memory persister
```

We are not working with a network yet to focus on the programming. POT JS behaves the same and is programmed-with the same whether it persists to memory or a network. Big KVSs will see a different latency. We will look into it later in this tutorial.

When creating a system using POT JS, from some point on it can be easier to develop with a network storage attached because it will keep everything stored across program restarts. The in-memory storage starts with a clean slate every time a script is restarted. For an earlier phase this will be more convenient.

Line ten of the log shows the binary form of what we are storing. The statement

```
await kvs.put("hello", "P.O.T.")
```

becomes

```
pot:  › ⟶  68656c6c6f000000000000000000000…: 03502e4f2e542e
```

In some cases, checking the binaries of what is really getting persisted can be helpful. For a rough idea of what is going on underneath: The key, **hello**, is stored as **68656c6c6f000000000000000…** A key, on the binary level, is zero-padded to 32 bytes. This is also the maximal length of a key. A longer key throws an error. **s.length** is your friend as it returns the byte length of a string, including for Unicode characters that may be stored as multiple bytes. Javascript uses UTF-16. **68 65 6c 6c 6f** is the hex (ASCII and) UTF-16 code for **hello**.

Keys can be **strings**, **numbers** or **Uint8Arrays**.

P.O.T. becomes **03502e4f2e542e**. The byte representation of **P.O.T** is prepended with a type byte: **03** stands for **string**. **50 2e 4f 2e 54 2e** is **P.O.T.** A value is not padded and can be 100,000 bytes or (much) longer (see pg. 8).

Values can be **strings**, **numbers**, **boolean**, **Uint8Arrays** or **null**. **Undefined** is returned for Keys that have no value stored in the KVS. **BigInt**, **symbols**, or **objects** are not supported (pg. 8).

#2 – MOVING TO THE BROWSER

The same code, pretty much, runs in the browser. Copy **t1.js** to **t2.html**.

```
cp t1.js t2.html
```

Replace the first line with the lines

```
<script src="lib/pot-web.js"></script>
<script>
```

If ever for a given project **pot.wasm** was not in the same folder as **pot-web.js**, a path tag can be used (see pg. 6).

To the end of the file add:

```
</script>
```

The file should now look like **potjs/tutorial/t2.html**:

```
<script src="potjs/lib/pot-web.js"></script>

<script>
; (async () => {
  await pot.ready()
  kvs = await pot.new()
  await kvs.put("hello", "P.O.T.")
  value = await kvs.get("hello")
  console.log("hello:", value)
})()
</script>
```

tutorial/t2.html

RUNNING IT

Run the file by using **`http-server`** or any other tool to serve the current directory:

```
npx http-server -c1 -o t2.html .
```

The -c1 parameter sets the cache to 1 second, the minimum. Don't trip yourself up with the browser's page cache when experimenting.

WHAT YOU SHOULD SEE

In the browser, nothing obviously, an empty page. But if you open the browser console:

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  hello: P.O.T.
pot:  › →  68656c6c6f0000000000000000000000…: 03502e4f2e542e
pot:  » get  hello: P.O.T.
pot:  › ←  68656c6c6f0000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

This is the exact same log as above, except for the second line, **`browser detected`**.

It is relevant for POT JS whether node.js or a browser is its sandbox, because the way that network access works is different for **`pot.wasm`** running in a browser vs. by node.js. Counter-intuitively, a WASM executable running in the browser has better access. But generally speaking, WASM was developed for the browser, and a browser script has a sandbox around itself that protects the machine.

#3 – SUB-COMPONENT INTEGRITY

To safeguard the integrity of the included script, fingerprints can be hardcoded into the HTML.

The required SHA-384 hash can be found in **potjs/lib/pot-web.js.sha384**. It looks like this:

```
cat potjs/lib/pot-web.js.sha384

Yb+faMKZRYWBwk2CrH9NeKRzg14w79Hyn7WCLdQbreD7coOaE+17RHMaPJv4zviS%
```

You will see a different hash, depending on the version of POT JS that you are using. There are pages online where you can verify the SHA-384.[11]

The **%** at the end is not part of the hash.

Copy **t2.html** to **t3.html** and add the **integrity** attribute. **"sha384-"** must be prepended to the hash for the **integrity** attribute.

The first line becomes (with or without line break):

```
<script src="potjs/lib/pot-web.js"
integrity="sha384-Yb+faMKZRYWBwk2CrH9NeKRzg14w79Hyn7WCLdQbreD7coOaE+17RHMaPJv4zviS">
```

head of tutorial/t3.html

The file will now look like **potjs/tutorial/t3.html**.

The code works exactly like before, showing the same results.

#4 & 5 – INTERACTION

We will stay with the browser and look at an interactive example. Create file **t4.html**.

```
<pre>

  key    <input id=key>
```

[11] E.g., https://emn178.github.io/online-tools/sha384.html

```
 value  <input id=value>

        <button onclick="put()"> put </button>

</pre>

<script src="potjs/lib/pot-web.js"></script>

<script>

 var kvs

 (async () => {
    await pot.ready()
    kvs = await pot.new()
 })()

 async function put() {
    key = document.getElementById('key').value
    value = document.getElementById('value').value
    await kvs.put(key, value)
 }

</script>
```

tutorial/t4.html

This source is filed under **potjs/tutorial/t4.html**.

The first lines are a web form that allows to input a key and a value.

```
<pre>

 key    <input id=key>

 value  <input id=value>

        <button onclick="put()"> put </button>

</pre>
```

part of tutorial/t4.html

Then comes the POT initialization, where a KVS is created.

```
<script src="potjs/lib/pot-web.js"></script>

<script>

 var kvs

 (async () => {
    await pot.ready()
    kvs = await pot.new()
})()
```

part of tutorial/t4.html

Then a **put()** function that reads from the form and stores the key-value pair to the KVS we created.

```
 async function put() {
    key = document.getElementById('key').value
    value = document.getElementById('value').value
    await kvs.put(key, value)
 }
```

part of tutorial/t4.html

RUNNING IT

Run the page with

```
npx http-server -c1 -o t4.html .
```

WHAT YOU SHOULD SEE

BROWSER

key

value

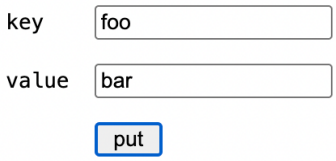
put

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
```

▶ Entering input for key and value, and hitting put:

BROWSER



BROWSER CONSOLE

```
...
pot:  » put  foo: bar
pot:  › ⟶  666f6f0000000000000000000000000…: 03626172
```

▶ We will now add the getting, which is analogous to the put.

To the form, add (before the </pre> tag):

```
<hr />

 key    <input id=search>

        <button onclick="get()"> get </button>

 value  <input id=result>
```

part of tutorial/t5.html

Add this get() function:

```
async function get() {
   key = document.getElementById ('search').value
   value = await kvs.get(key)
   document.getElementById ('result').value = value
}
```

part of tutorial/t5.html

The file should now look like **potjs/tutorial/t5.html**:

```
<pre>

 key    <input id=key>

 value  <input id=value>

        <button onclick="put()"> put </button>

<hr />

 key    <input id=search>

        <button onclick="get()"> get </button>

 value  <input id=result>

</pre>

<script src="potjs/lib/pot-web.js"></script>

<script>

 var kvs

 (async () => {
    await pot.ready()
    kvs = await pot.new()
 })()

 const field = (id) => { return document.getElementById(id) }

 async function put() {
```

```
    key = field('key').value
    value = field('value').value
    await kvs.put(key, value)
  }

  async function get() {
    key = document.getElementById ('search').value
    value = await kvs.get(key)
    document.getElementById ('result').value = value
  }

</script>
```

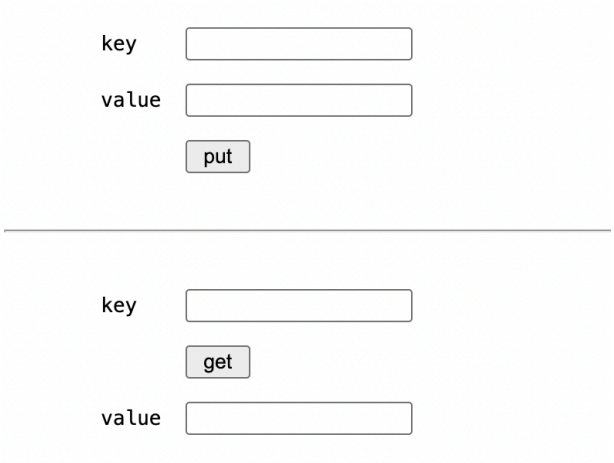
tutorial/t5.html

RUNNING IT

```
npx http-server -c1 -o t5.html .
```

WHAT YOU SHOULD SEE

BROWSER



BROWSER CONSOLE

As before.

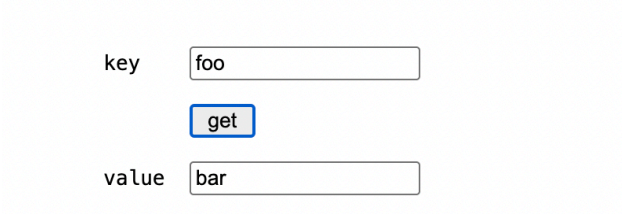
▶ Entering **foo**, **bar** and putting it to the KVS will also show the same browser console log as before:

```
...
pot:  » put  foo: bar
```

```
pot:  › ⟶  666f6f0000000000000000000000000…: 03626172
```

▶ Entering **foo** in the third input field, labeled **key**, and hit **get**, will result in the value field displaying **bar**:

BROWSER



BROWSER CONSOLE

```
...
pot:  » get  foo: bar
pot:  › ⟵  666f6f0000000000000000000000000…: 03626172
```

ERROR CONDITIONS

XXX > The WASM executable is not a library but a Go program that runs continually, waiting for the next request. If it crashes, the functionality it extends goes away. The main methods of JS objects that POT JS defines become unavailable.

It is relatively easy to deadlock the Go WASM by using the synchronous functions in the wrong context. Those are the POT JS functions ending on **Sync**. They are generally not safe to use when using a Swarm network as storage backend in the browser. But they work well for in-memory experimenting, and in any case with node.js.

In case the Go WASM program goes down, it has to be re-initialized. < XXX

#8 – VERBOSITY

The default logging to screen is relatively detailed. If you want to see less or nothing at all, the level of logging can be changed with a value from 0 to 5, zero meaning no logging at all, 4 what you saw in the terminal up to now. The levels have names:

#	name	description
0	`pot.NONE`	no logging, not even errors.
1	`pot.CRITICAL`	logs only errors that appear to arise from a POT JS malfunction.
2	`pot.ERROR`	logs all programming and runtime errors.
3	`pot.INFO`	general runtime information is logged, close to what you saw so far.
4	`pot.DEBUG`	certain data, like binary keys and values are logged. The default.
5	`pot.TRACE`	very specific program paths and minor settings are logged.

The verbosity level is set using `pot.setVerbosity()`.

To try it, copy t5.html to t8.html and add the verbosity-setting call to level `INFO` after `pot.ready()`:

```
(async () => {
    await pot.ready()
    pot.setVerbosity(pot.INFO)
    kvs = await pot.new()
 })()
```

part of tutorial/t8.html

RUNNING IT

```
npx http-server -c1 -o t8.html .
```

▶ Trying the same put and get as before (`foo`-`bar` put, `foo` get), the lowest level log lines are gone.

WHAT YOU SHOULD SEE

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
```

```
pot:  » put  foo: bar
pot:  » get  foo: bar
```

log of tutorial/t8.html – with verbosity setting pot.INFO

If you look closely, the lines that had a single angle › are now not logged any more. They were DEBUG level. The ones with a double angle remain. They are INFO level.

If you ever need number displayed unabbreviated, use TRACE.

▶ Setting verbosity to pot.NONE results in this output.

Four lines remain, even with pot.NONE:

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
```

log of altered tutorial/t8.html – with verbosity setting pot.NONE

To eliminate those remaining four lines, a different method has the be used: a verbosity attribute can be added to the script tag that sources pot-web.js:

```
<script src="potjs/lib/pot-web.js" verbosity=2></script>
```

part of tutorial/t9.html

This verbosity setting suppresses even the first four lines coming from POT JS.

If you try it, there will be no logging in the browser console.

Setting 2, meaning ERROR, is enough for this, lower would suppress even error logs. But not their being thrown and popping up otherwise if you don't catch them.

The value assigned to the verbosity attribute must be a number – or a literal or a variable you defined – because pot.NONE etc. do not exist yet at this point yet. The initialization of WASM creates them.

For a node.js script, to likewise set the verbosity at the earliest, an additional parameter can be added to the require call, or a variable globalThis.potjs_verbosity set (see pg. 13).

#9 – NODE APP

Switching back to node.js, we will now briefly look at a single-script web app that works exactly like the form we just created with the difference that the POT JS logic runs on the server instead of in the browser.

As you will see, specifically the POT code is as little and looks pretty much the same. There are no surprises.

The script looks like this. We will talk through it immediately:

```
const http = require("http")
const qs = require("querystring")
require("./lib/pot-node")

const form = `<pre>

    <form method=post action="http://localhost:3000">

    key    <input name=key>

    value  <input name=value>

           <input type=submit name=button value=put>


 <hr />


    key    <input name=search>

           <input type=submit name=button value=get>

    value  <input name=result>

    </form>

 </pre>`

var kvs

; (async () => {
 await pot.ready()
 kvs = pot.newSync()
```

```
})()

async function put(key, value) {
 await kvs.put(key, value)
 return `put ${key}: ${value}`
}

async function get(search) {
 value = await kvs.get(search)
 return `get ${search}: ${value}
    <script>
    document.getElementsByName('result')[0].value = '${value}'
    </script>`
}

const server = http.createServer(function(request, response) {

 if (request.method == 'POST') {

    var body = ''
    request.on('data', function(data) {
           body += data
    })

    request.on('end', async function() {

           var log
           const post = qs.parse(body)

           if(post.button == 'put')
                  log = await put(post.key, post.value)
           else
                  log = await get(post.search)

           console.log("srv: " + log.match(/.*/)[0])

           response.writeHead(200, {'Content-Type': 'text/html'})
           response.end(form + '<hr><br><pre>\t\t' + log)
    })
 }
 else {
    response.writeHead(200, {'Content-Type': 'text/html'})
    response.end(form)
 }
})
```

```
  .listen(3000, '127.0.0.1')

  console.log(`srv: listening at http://127.0.0.1:3000`)
```

tutorial/t11.js

RUNNING IT

```
  node t11.js
```

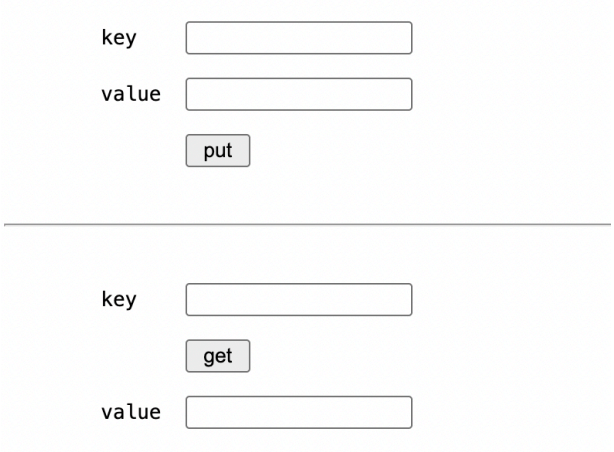
WHAT YOU SHOULD SEE

TERMINAL

```
  srv: listening at http://127.0.0.1:3000
  pot:  » POTWASM
  pot:  » node.js detected
  pot:  » init done
  pot:  » ready
  pot:  › created new in-memory persister
  pot:  » (re)-using in-memory persister
  pot:  » initialize new P.O.T. - slot 1
  pot:  › slot ref: 1
```

BROWSER

▶ Open http://127.0.0.1:3000 (you have to open the URL yourself this time) you should see the same picture as in the previous tutorial:



screen of tutorial/t11.js – http://localhost:3000

▶ Entering the same input – **foo**, **bar**, put; **foo** get – should result in these familiar logs:

```
pot:  » put  foo: bar
pot:  › ⟶  666f6f0000000000000000000000000…: 03626172
srv: put foo: bar
pot:  » get  foo: bar
pot:  › ⟵  666f6f0000000000000000000000000…: 03626172
srv: get foo: bar
```

POT JS behaves the same, whether it is running in the browser or on the server.

#12 – PERSISTENCE

To connect with a network, so the KVSs are stored on it, we have to add the following.

new() has to be supplied with a URL to a Swarm bee node, and a batch ID:

```
kvs = await pot.new(swarm_url, batch_id)
```

And after any **put()**, at some point, there should be a **save()**. We put it immediately after.

```
  await kvs.save()
```

The Swarm URL and batch ID need to come from somewhere, we'll read them from the command line (lines 5 and 6 in the following). The resulting script is in tutorial/t12.js:

```
require("./potjs/lib/pot-node")

; (async () => {
  await pot.ready()
  swarm_url = process.argv[2]
  batch_id = process.argv[3]
  kvs = await pot.new(swarm_url, batch_id)
  await kvs.put("hello", "P.O.T.")
  await kvs.save()
  value = await kvs.get("hello")
  console.log("hello:", value)
```

```
})()
```

tutorial/t12.js

Such script can interact with Swarm. But for our tests, we are setting up a local Swarm test network on Docker and connect to that. Not least because we don't need real Bzz tokens for it.

Install the Swarm CLI, and the test network provided by fairdata society.

```
npm install -g @ethersphere/swarm-cli
swarm-cli --version
npm install -g @fairdatasociety/fdp-play
fdp-play --version
```

Start Docker and then spin up the network:

```
fdp-play start -detach
```

You should see:

```
✓ Network is up
✓ Blockchain node is up and listening
✓ Queen node is up and listening
✓ Worker nodes are up and listening
```

Next, buy some (test) Swarm postage stamps – the batch ID – using the prepopulated accounts of the test network. For use with Swarm mainnet, you will need Bzz.

```
% swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b
```

As the warning warns, the batch creation takes some minutes.

```
Estimated cost: 0.1048576000000000 xBZZ
Estimated capacity: 599.775 MB
Estimated TTL: 2 days
Type: Immutable
At full capacity, an immutable stamp no longer allows new content uploads.
Stamp ID: e2a17af2c560cfaa7229309f1f5b59f0d635dfab8f5e604b8239b3719b51ad82
```

Copy the batch ID (`Stamp ID`) that is displayed for you – the hex digits in the last line – and put them last in the command line that calls the node.js script (below). The other parameter is `http://localhost:1633`. That is where the local Swarm test network API is found.

RUNNING IT

```
node t12.js http://localhost:1633 \
e2a17af2c560cfaa7229309f1f5b59f0d635dfab8f5e604b8239b3719b51ad82
```

WHAT YOU SHOULD SEE

TERMINAL

```
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › using bee url : ‹http://localhost:1633›
pot:  › using batch id: ‹e2a17af2c560cfaa7229309f1f5b59f0…›
pot:  » created new hybrid swarm network loader
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  hello: P.O.T.
pot:  › ⟶  68656c6c6f0000000000000000000000…: 03502e4f2e542e
pot:  » saving storage
pot:  › ref32: b77d8826af7c1684ae88bb63afdee7e3…
pot:  » get  hello: P.O.T.
pot:  › ⟵  68656c6c6f0000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

And that's it. Now let's try to read the stored values.

Using a network rather than in-memory storage, the values we store persist.

To read a KVS back from the storage, we need its save reference. It is cut off in the log at this point. To see it all, we amp up the velocity level to TRACE before the save – as a demonstration as much as a quick fix.

```
pot.setVerbosity(pot.TRACE)
```

part of tutorial/t13.js

Running it again, the full save reference is visible:

```
pot:  » saving storage
pot:  › ref32: b77d8826af7c1684ae88bb63afdee7e3b81bcb6d0ee15a4d1aa096452dcb7624
```

Note that it is the same reference as the truncated one we saw in the log before. Even though a new map had been created and the value stored and saved anew.

The reason is that the same content will result in the save reference always. Because the save reference is actually the root of the proximity order tree (POT).

Now, to learn about retrieving values, create a file t14.js:

```
require("./potjs/lib/pot-node")

; (async () => {
        await pot.ready()
        swarm_url = process.argv[2]
        batch_id = process.argv[3]
        save_ref = process.argv[4]
        kvs = await pot.load(save_ref, swarm_url, batch_id)
        value = await kvs.get("hello")
        console.log("hello:", value)
})()
```

To explain the changes: 1) a third parameter is being read from the command line

```
        save_ref = process.argv[4]
```

And 2) `new()` is replaced by `load()`, which takes an extra parameter, the save reference.

```
        kvs = await pot.load(save_ref, swarm_url, batch_id)
```

The `put()` and `save()` calls are deleted.

If we find any value for the key `hello` this time, it will be from the network, i.e., what we stored the last time.

RUNNING IT

The parameters are in order, the swarm URL, the postage batch ID and the save reference.

```
node t14.js http://localhost:1633 \
e2a17af2c560cfaa7229309f1f5b59f0d635dfab8f5e604b8239b3719b51ad82 \
b77d8826af7c1684ae88bb63afdee7e3b81bcb6d0ee15a4d1aa096452dcb7624
```

WHAT YOU SHOULD SEE

```
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  » load P.O.T. by reference
pot:  › reference: b77d8826af7c1684ae88bb63afdee7e3…
pot:  › using bee url : ‹http://localhost:1633›
pot:  › using batch id: ‹e2a17af2c560cfaa7229309f1f5b59f0…›
pot:  » created new hybrid swarm network loader
pot:  › slot ref: 1
pot:  » get  hello: P.O.T.
pot:  › ←  68656c6c6f0000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

The value `P.O.T.` is found for the key `hello`, loaded from the local Swarm network.

Note that what doesn't work is to assume that the save reference were a handle for the key value store as it evolves. It is a handle for the store only with the key-values exactly as when it was saved. When it is changed and saved again, a different save reference would return. It's the root of a state trie.

But if the same sequence of updates is run, it will result in the same save reference again when saved.

◊

This concludes the tutorial. Check out Further Reading (pg. 16).

Functions not changed on this high level.
Functions not changed on this high level.
New functions added and names completely reworked.
No changes to functions.
Support functions added.
No changes to functions.
Some support functions added.

FUNCTION OVERVIEW

The main functions are split in two-by-two main groups: asynchronous / synchronous and typed / raw respectively. This makes for four different ways of calling but only one group – asynchronous typed calls – is relevant in most cases. Additionally, there are initialization, test and support functions.

The most relevant functions are `put()` and `get()`, which are asynchronous, typed functions. That means, `get()` returns a promise for the value to be retrieved and it will automatically be in the type that it was stored with using `put()`.

Asynchronous functions return promises and throw an error on failuire. Synchronous functions block and return a value, error or null. They don't throw errors. The synchronous functions' names end on `Sync`. Synchronous functions can be handy for prototyping and special situations but in the browser generally only work for tests using the in-memory persistence, not when connecting to Swarm or a test network. They don't have a problem when using node.js. It is possible that they might work well with the background threads of Web Workers but this has not been tested. Therefore, they should simply not be used in production in the browser. The asynchronous functions work in all circumstances.

Typed functions – the default – mark the Javascript type of a value in the storage, so as to automatically return the right type on retrieval. This can be a boolean, a number,[12] or a string. Raw functions will usually take an Uint8Array of bytes as argument to store it as is and the right function (`getRaw`, `getBoolean`, `getString`, `getNumber`) has to be called later to get the expected type back out.. The raw functions have `Raw` or a type as name component. Raw functions will be useful in special situations, whenever binary data is handled, but usually the vanilla typed, asynchronous functions `new`, `save`, `put` and `get` will be the first choice.

For more details than given below, and examples, see the function reference (separate document).

MAP INITIALIZATION

`pot.new`	create a new Swarm key-value-store, i.e., a map, async
`pot.newByReference`	create a new Swarm map from a reference of a prior save, async
`map.save`	store a Swarm KVS to the network, async
`pot.newSync`	create a new Swarm key-value-store, i.e., a map, sync
`pot.newByReferenceSync`	create a new Swarm map from a reference of a prior save, sync
`map.saveSync`	store a Swarm KVS to the network, sync

TYPE-AWARE

`put`	return a promise for null or Error. Asynchronous.
`get`	return a promise for the typed get, the type will be as put. Async.
`putSync`	return a promise for null or Error. Synchronous.
`getSync`	return a promise for the typed get, i.e., the type will be as put. Sync.

[12] Javascript does not have native integers and stores all numbers as IEEE 754 64-bit floats internally.

RAW BYTE ACCESS

`putRaw`	put a raw **Uint8Bytes** array, async.
`getRaw`	get a raw **Uint8Bytes** array, async.
`getBoolean`	get the raw value as Boolean. First byte 0 for false, all else true, async.
`getNumber`	get the raw value as floating point number. Must be 8 bytes. Async.
`getString`	get the raw value as string, async.
`putRawSync`	put a raw **Uint8Bytes** array, synchronous.
`getRawSync`	get a raw **Uint8Bytes** array, synchronous.
`getBooleanSync`	get the raw value as Boolean. First byte 0 for false, all else true, sync.
`getNumberSync`	get the raw value as floating point number. Must be 8 bytes. Sync.
`getStringSync`	get the raw value as string, synchronous.

POT INITIALIZATION

`pot.ready`	promise that resolves when **pot.wasm** is initialized.
`onPotInitialized`	called by **pot.wasm** when it is initialized.
`pot.setOptimization`	controls whether JS function resources are recycled.
`pot.setVerbosity`	controls the extent to which POT JS logs, a value from 0 to 5.
`pot.setValueSizeLimit`	cap of value byte size. Can be set to any number.

`globalThis.potjs_optimization`	setting optimization before initialization
`globalThis.potjs_verbosity`	setting verbosity before initialization

TESTING AND SUPPORT

These functions are not used in production. They serve to test the integrity of POT JS. Some are noops when compiled for production

`log`	write to browser console via Go to verify a language-land roundtrip
`hello`	write hello to browser console to verify established WASM stream
`randKey`	32 random bytes for key testing
`randValue`	79 to 101 random bytes for value testing (range from Go POT)
`randBuffer`	arbitrary amount of random bytes for value testing
`typeEncodedBytes`	JS type detection and value encoding with correct leading type byte
`typeDecodedValue`	JS type detection of raw kvs value by leading type byte
`hangingPromise`	blocking promise (in Go) with time out for exception testing
`panickingPromise`	promise (in Go) that panics internally to test intra-promise recovery
`setFail`	trigger the next mock storage function called to return an error
`setPanic`	trigger the next mock storage function called to raise a panic
`setDelay`	trigger the next mock storage function called to stall for a given time
`setHang`	trigger the next mock storage function called to stall indefinitely

EXAMPLES

This chapter lists and explains ten POT JS code examples.

For a more hands-on explanations of how to program POT JS, see the Tutorial chapter (pg. 17).

To run the first example, serve the main project directory and open `examples/example1.html`:

```
npx http-server -o examples/example1.html .
```

or use

```
make example1
```

from `potjs/`, not from `examples/`.

Examples 5, 6, and nine are started with a number of parameters. It makes sense to use make.

There are nine examples, 1-9, that can be started this way: XXX vvv correct overview? vvv

`example1.html`	**minimal** in-browser example as above, in-memory storage
`example2.html`	**interactive** in-browser example with input fields, integrity
`example3.html`	like example 2 without using pot-web.js
`example4.html`	like example 1 using **synchronous** access functions
`example5.html`	like example 1 but with local **Swarm** network storage
`example6.html`	like example 5, parametrized, with WASM path and log level 2
`example7.js`	like example 1 but for **node.js**
`example8.js`	similar to example 2, interactive web app for node.js, in-mem.
`example9.js`	identical to example 8, called to use a local Swarm network

EXAMPLE 1: MINIMAL

A simple put and get, logging to browser console. See previous page on how to run it.

- **in-browser** (code that is for node.js instead uses **`pot-node.js`**, not **`pot-web.js`**)
- **in-memory** (to not be in-memory, **`pot.new()`** would need to receive arguments, see above.)
- **asynchronous** (asynchronous functions all have synchronous corollaries that end in **`Sync()`**.)

`examples/example1.html`

```
<script src="lib/pot-web.js"></script>
<script>
(async () => {
      await pot.ready()
      map = await pot.new()
      await map.put("hello", "POT!")
      value = await map.get("hello")
      console.log("key ‹hello› has:", value)
})()
</script>
```

code of examples/example1.html

WHAT YOU SHOULD SEE

BROWSER

Empty, white page (or, if you used **`example1.html`**, it says "POT JS Example 1: Hello – Check source and console.") The relevant information is in the browser console.

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  hello: POT!
pot:  › ⟶  68656c6c6f000000000000000000000…: 03504f5421
pot:  » get  hello: POT!
pot:  › ⟵  68656c6c6f000000000000000000000…: 03504f5421
key ‹hello› has: POT!
```

EXAMPLE 2: INTERACTION

Interactive use, verbosity setting and integrity checks: a html form accepts input and shows results. The action is handled entirely in the browser. The sample KVS is stored in an in-memory storage.

The **script** tag in this example has three additional attributes. The first one, **wasm**, is the path to the **pot.wasm** file. It is (beyond its demonstration) unnecessary in this case as **pot.wasm** is looked for by default in the same folder as **pot-web.js**.

The example also shows how to set verbosity in the script tag, using the **verbosity** attribute. See pg.13 for the list of valid arguments. In the instance it is set to level 2, which is INFO level, producing fewer log lines.

The **integrity** attribute accepts a hash of the included script **pot-web.js** to verify that the correct file arrives in the browser.

- in-browser
- in-memory
- asynchronous

examples/example2.html

```
<pre>

    POT JS Example 2: Interactive & Integrity

    key    <input id=key>
    value  <input id=value>
           <button onclick="put()"> put </button>

<hr />

    key    <input id=search>
           <button onclick="get()"> get </button>
    value  <input id=result>

    <img src=favicon.ico>

</pre>

<script src="lib/pot-web.js" wasm="lib/pot.wasm" verbosity="3" integrity="sha384-
4hEVHN8uJHWpa4gAFHswdsKiz5Ss9sFjp/X+HLrZGsqVsQunkRmMdFojIZ38s64T"></script>

<script>
```

```
	var kvs

	(async () => {
		await pot.ready()
		kvs = await pot.new()
	})()

	const field = (id) => { return document.getElementById(id) }

	async function put() {
		key = field('key').value
		value = field('value').value
		await kvs.put(key, value)
	}

	async function get() {
		key = field('search').value
		value = await kvs.get(key)
		field('result').value = value
	}

</script>
```

examples/example2.html

RUNNING IT

```
	npx http-server -o examples/example2.html .
```

or use

```
make example2
```

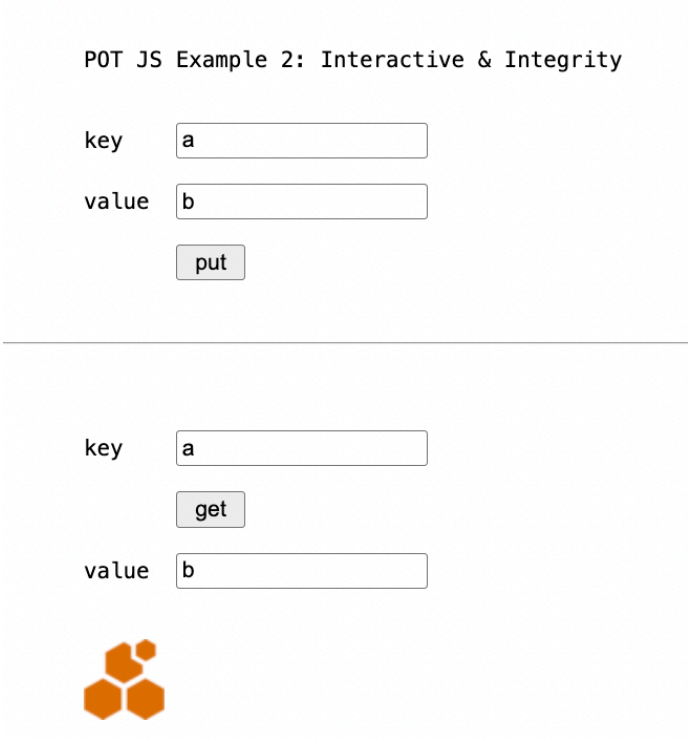
1) Enter a value for **key** and **value**, and hit **put**.

2) Enter the same **key** again in the lower screen area and hit **get**.

This will yield the **value** in the lowest field.

WHAT YOU SHOULD SEE

BROWSER



screen of example 2

BROWSER CONSOLE

Because verbosity was set to 3 in the script attribute, there are fewer details shown than in example 1. The binary key and value hex digits are suppressed to unclutter the picture.

But 3 means INFO. This is still a chatty setting.

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  » put  a: b
pot:  » get  a: b
```

EXAMPLE 3: MANUAL INITIALIZATION

This examples showcases a more explicit initialization, leaving out the Javascript wrapper: it does not use **pot-web.js** but initializes **pot.wasm** directly.[13] It is not functionally different from example 2.

Instead of the script tag attribute **verbosity**, it uses **globalThis.potjs_verbosity** to set the log level.

To be perfect, this example should have a mechanism that would prevent its **get()** and **put()** functions to be called before WASM initialization is complete (cf. xxx). In practice, the buttons in this form will never be pressed fast enough by a user to create a race condition. However, cleaner code would use **onPotInitialized()** (see pg. xxx).

- in-browser
- in-memory
- asynchronous

examples/example3.html

```
...

    const go = new Go()
    window.potjs_verbosity = 3
    WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
            .then(async (r)=>{
                    go.run(r.instance)
                    kvs = await pot.new()
            })
...
```

part of examples/example3.html

WHAT YOU SHOULD SEE

The same as before in example 2.

[13] Note that the corollary is not possible for node.js with network storage because that needs a Javascript fetch function out of **pot-node.js**.

EXAMPLE 4: SYNCHRONOUS

This example – that is otherwise like example 1 – uses synchronous calls that can be helpful for prototyping but don't work connected to a network in the browser. They work fine in every circumstance with node.js.

- in-browser
- in-memory
- synchronous

examples/example4.html

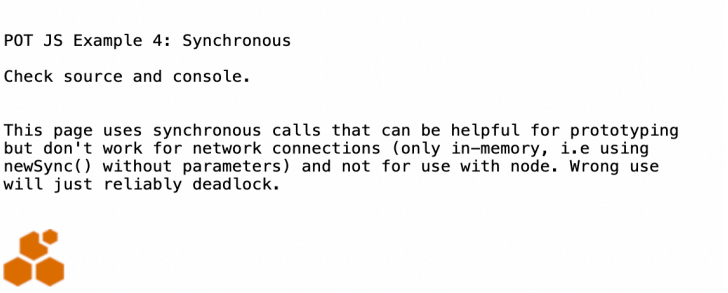
```
...
<script src="lib/pot-web.js"></script>
<script>
function onPotInitialized() {
    kvs = pot.newSync()
    kvs.putSync("hello", "POT!")
    value = kvs.getSync("hello")
    console.log("key ‹hello› has:", value)
}
</script>
```

core of examples/example4.html

WHAT YOU SHOULD SEE

Almost the same as before in example 1.

BROWSER



BROWSER CONSOLE

The same as before in example 1.

EXAMPLE 5: NETWORK

This page connects to a local Swarm network instead of using in-memory storage.

It is in principle the same as example 1, it just supplies the relevant network parameters to `pot.new()`.

The source code below is for illustration: the `batch id` would have to be updated to run it. But `make` does that for you, it works when using `make example5`. That patches the source code of `example/example5.html` with the latest batch id.

- in-browser
- network persistence
- asynchronous

examples/example5.html

```
<pre>

    POT JS Example 5: Network
    Check source and browser console.

    This page connects to a local Swarm network.

    It persists a value, and does not read it back. put() stores to
    cache, save() stores to the network.

    To test it, use `make example5`, which will start a local Swarm
    network and update the batch id literal in the pot.new() call
    below after creating a new batch. Note that some other make rules
    will re-use this batch id, stored in .batch_id, if it exists.

    <img src=favicon.ico>

</pre>

<script src="lib/pot-web.js"></script>
<script>

(async () => {
    await pot.ready()
    kvs = await pot.new("http://localhost:1633",
"12c073a8b2a8f94b3013d1794e6ad713905d4083503b2180a798e2d5841c6666")
```

```
    await kvs.put("hello", "POT!")
    ref = await kvs.save()
    console.log("tree root is:", ref)
})()

</script>
```

RUNNING IT

This example is more involved, it needs a docker engine and fairdata'a Swarm test network installed, as well as the Swarm CLI. A postage batch (see pg. xxx) needs to be created, and, as noted above, the id patched into examples/example5.html.

Install docker, and do

```
npm install --global @ethersphere/swarm-cli
npm install --global @fairdatasociety/fdp-play
make example5
```

The creation of the postage batch takes some minutes. For more details on it see the instructions, (pg. xxx) and tutorial (pg. xxx).

WHAT YOU SHOULD SEE

TERMINAL

```
% make example5
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
server still running
/Library/Developer/CommandLineTools/usr/bin/make locnet_start
⬡ starting five local swarm nodes
on failure despite docker running, try erasing existing fdp-play containers
fdp-play start --detach
All containers are up and running
rm -f .batch_id
/Library/Developer/CommandLineTools/usr/bin/make .batch_id
⬡ buy stamps (free in this test setup)
swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b | tee .batch_creation
Estimated cost: 0.1048576000000000 xBZZ
Estimated capacity: 599.775 MB
Estimated TTL: 2 days
Type: Immutable
At full capacity, an immutable stamp no longer allows new content uploads.
```

```
Stamp ID: 37004b73068c18e024d8221ff55b65847ba7d63b739ff44785ec233d0163dced
grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
⬡ postage batch ID: 37004b73068c18e024d8221ff55b65847ba7d63b739ff44785ec233d0163dced
sed -i.bak -e "s/\(pot\.new.*\)\"[0-9a-fA-F]*\")/\1\"$(cat .batch_id)\")/"
examples/example5.html ; rm -f examples/example5.html.bak
open http://127.0.0.1:8080/examples/example5.html
... % ...  "GET /examples/example5.html" ...
...  "GET /examples/lib/pot-web.js" ...
...  "GET /examples/favicon.ico" ...
...  "GET /examples/lib/pot.wasm" ...
```

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › using bee url : ‹http://localhost:1633›
pot:  › using batch id: ‹37004b73068c18e024d8221ff55b6584…›
pot:  » created new generic swarm network loader
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  hello: POT!
pot:  › ⟶  68656c6c6f0000000000000000000000…: 03504f5421
pot:  » saving storage
pot:  › ref32: 5968b03183efcb859b135c21e5e98b37…
tree root is: 5968b03183efcb859b135c21e5e98b378e189789cfa3593761a74b69fca0aed3
```

EXAMPLE 6: NETWORK II

Example 6 works in essence like example 5 but receives the batch id as a parameter, and demonstrates how a KVS is persisted and then a new KVS object instantiated for it, using `save()` and `load()`.

Regarding the meaning of the save reference, see xxx. In a nutshell, kvs1 and kvs2 are not two different key-value-stores but they are also not the same. The save reference means a certain state, not a certain store's identity. Every store that has the same contents will have the same save reference.

- in-browser
- network persistence
- asynchronous

examples/example6.html

```
<pre>
     POT JS Example 6: Network II

     Check source and browser console.

     This page connects to a local Swarm network.

     It can be tried by running `make example6`, which takes care
     of the local network and the batch id. The script connects to
     the http server at port 8080, the Swarm queen node at 1633.

     What those make rules do is essentially:

       fdp-play start --detach
       swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b > .batch_creation
       grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
       npx http-server -c1 . &
       open "http://127.0.0.1:8080/example6.html?bee=http://127.0.0.1:1633&batch=$(cat .batch_id)"

     If the batch id file exists, or the http server is already running, the
     make rules will skip the respective lines.

     Use `make stop` to stop the servers, or keep them running for the next
     examples.

     <img src=favicon.ico>

</pre>

<script src="lib/pot-web.js" verbosity=2></script>
<script>

(async () => {

     await pot.ready()

     batch = new URLSearchParams(window.location.search).get('batch');

     kvs1 = await pot.new("http://localhost:1633", batch)
     console.log("kvs #1 -- put hello      : POT!")
     await kvs1.put("hello", "POT!")
     value = await kvs1.get("hello")
     console.log("kvs #1 -- key ‹hello› has:", value)
     ref1 = await kvs1.save()
     console.log("kvs #1 -- trie root is   :", ref1)

     kvs2 = await pot.load(ref1, "http://localhost:1633", batch)
     value2 = await kvs2.get("hello")
     console.log("kvs #2 -- key ‹hello› has:", value2)
     ref2 = await kvs2.save()
     console.log("kvs #2 -- trie root is   :", ref2)

     console.log("kvs #2 -- put baz        : true")
     await kvs2.put("baz", true)
     ref3 = await kvs2.save()
     console.log("kvs #2 -- trie root is   :", ref3)

})()
</script>
```

examples/example6.html

RUNNING IT

This example uses the setup of example 5 – the local network, the postage batch etc. Set up example 5 to prepare for this. Then do

```
make example6
```

WHAT YOU SHOULD SEE

TERMINAL

```
% make example6
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
server still running
/Library/Developer/CommandLineTools/usr/bin/make locnet_start
⬡ starting five local swarm nodes
on failure despite docker running, try erasing existing fdp-play containers
fdp-play start --detach
All containers are up and running
/Library/Developer/CommandLineTools/usr/bin/make .batch_id
make[1]: `.batch_id' is up to date.
open "http://127.0.0.1:8080/examples/example6.html?bee=http://127.0.0.1:1633&batch=$(cat
.batch_id)"
... % ... "GET /examples/example6.html?bee=http://127.0.0.1:1633&batch=37004b73068c1...
... "GET /examples/favicon.ico" ...
... "GET /examples/lib/pot-web.js" ...
... "GET /examples/lib/pot.wasm" ...
```

BROWSER

Some explanations.

BROWSER CONSOLE

Verbosity is set to 2, which effects that beyond the console.log() messages from the example code, only error messages would be shown from the POT JS engine.

```
kvs #1 -- put hello      : POT!
kvs #1 -- key ‹hello› has: POT!
kvs #1 -- trie root is   : 5968b03183efcb859b135c21e5e98b378e189789cfa3593761a74b69fca0aed3
kvs #2 -- key ‹hello› has: POT!
kvs #2 -- trie root is   : 5968b03183efcb859b135c21e5e98b378e189789cfa3593761a74b69fca0aed3
kvs #2 -- put baz        : true
kvs #2 -- trie root is   : 92bc2a4cf5b67780bfa289ddf32b558c11cf88d53d8c793171d97bc2442a22dc
```

EXAMPLE 7: NODE

This example shows how POT JS works with node.js. Pretty much like in the browser.

The one difference vs. in-browser use is that **pot-node.js** is required instead of **pot-web.js**.

- node.js
- in-memory
- asynchronous

examples/example7.js

```
console.log(`

    POT JS Example 7: node.js
    Check source and browser console.

`)

require("./lib/pot-node")

; (async () => {
    await pot.ready()
    kvs = await pot.new()
    await kvs.put("hello", "node")
    value = await kvs.get("hello")
    console.log("hello:", value)
})()
```

RUNNING IT

This example is plain and can be run without any further preparation by

```
node examples/example7.js
```

or

```
make example7
```

WHAT YOU SHOULD SEE

Essentially the same as in example 1, just on the terminal instead the browser / console.

TERMINAL

```
% node examples/example7.js

     POT JS Example 7: node.js
     Check source and browser console.

pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  hello: node
pot:  › ⟶  68656c6c6f0000000000000000000000…: 036e6f6465
pot:  » get  hello: node
pot:  › ⟵  68656c6c6f0000000000000000000000…: 036e6f6465
hello: node
```

EXAMPLES 8: NODE WEB APP

This example does the same as example 2, but this time, the logic runs on the server side, node.js, instead of in the browser. The result is very similar, so is the source code.

The top is the HTML that is sent to the browser. Then follow the initialization of POT and the **`get()`** and **`put()`** function that the buttons in the web form will call. This time not via Javascript in the browser but by HTML POSTs to the server, where these **`get()`** and **`put()`** reside. It deserves repeating because it looks so similar to example 2. But they run at a different time, connected to the web page in a different way. The last part is the web server.

In effect this is a miniature single-source web app. The page is served from the server script, rather than a file. The POT functions run on the server, not in the browser.

The script also demonstrates the use of **`WebAssembly.instantiate()`** for node.js, leaving **`pot-node.js`** out. In rare instances this might be useful.

- node.js
- in-memory
- asynchronous

examples/example8.js

```
console.log(`

    POT JS Example 8: Node.js, in-memory, explict init, asynchronous

    This is a self-contained web app, serving a page to receive input,
    returning the results.

    open http://localhost:3000
`)

const http = require("http")
const qs = require("querystring")
const fs = require("fs")
require("./lib/wasm_exec")

const form = `<pre>

            POT JS Example 8: Node.js Web App Server

            <form method=post action="http://localhost:3000">

            key    <input name=key>
            value  <input name=value>
                   <input type=submit name=button value=put>
    <hr />
            key    <input name=search>
                   <input type=submit name=button value=get>
            value  <input name=result>

            </form>

    <hr />

    </pre>

    <script> console.log('see server log in terminal') </script>

    <pre>
            `
var kvs
var go = new Go()

WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
    .then((r) => { go.run(r.instance) })

global.onPotInitialized = async () => {

    bee = process.argv[2]
    batch = process.argv[3]
    kvs = await pot.new(bee, batch)
```

```
    // There is no catching of early calls of put() and get()
    // in this example, which in theory could race the loading
    // of pot.wasm and the creation of kvs.
}

async function put(key, value) {
    await kvs.put(key, value)
    return `put ${key}: ${value}`
}

async function get(search) {
    value = await kvs.get(search)
    return `get ${search}: ${value}

            <script>
            document.getElementsByName('result')[0].value = '${value}'
            </script>`
}

const server = http.createServer((request, response) => {

    var log  = ''
    var body = ''

    request.on('data', (data) => body += data)
           .on('end', async () => {
                  const { button, key, value, search } = qs.parse(body)
                  switch(button) {
                  case 'put': log = await put(key, value) ; break
                  case 'get': log = await get(search)
                  }
                  response.writeHead(200)
                  response.end(form + log)
    })
})
.listen(3000, '127.0.0.1')

console.log('srv: listening at http://127.0.0.1:3000')
```

examples/example8.js

RUNNING IT

This example can be run without any further preparation by

```
node examples/example8.js
```

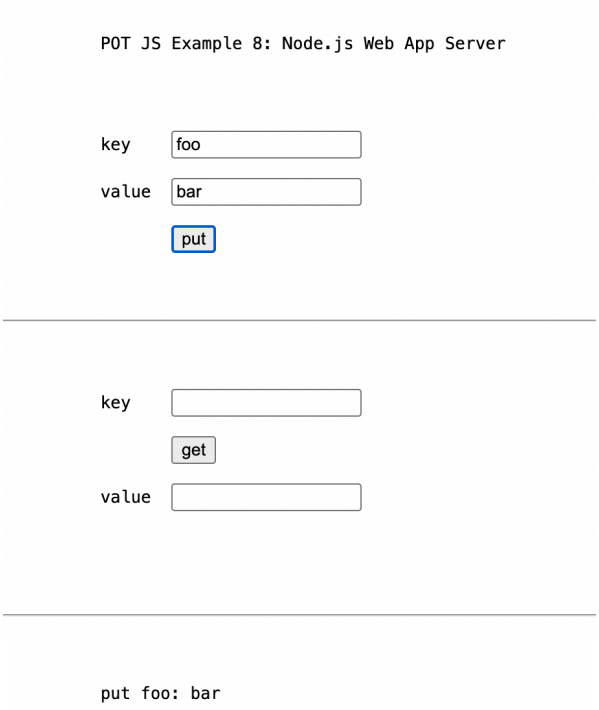
or

```
make example8
```

WHAT YOU SHOULD SEE

BROWSER

The screen will refresh with fields empty on every button press.



montage of example 8 before and after pressing put

TERMINAL

```
% make example8
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
● √ pot.wasm built for production
open http://127.0.0.1:3000
node examples/example8.js

     POT JS Example 8: Node.js, in-memory, explict init, asynchronous

     This is a self-contained web app, serving a page to receive input,
     returning the results.

     open http://localhost:3000

srv: listening at http://127.0.0.1:3000
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  foo: bar
pot:  › ⟶  666f6f0000000000000000000000000…: 03626172
pot:  » get  foo: bar
pot:  › ⟵  666f6f0000000000000000000000000…: 03626172
```

EXAMPLE 9: NODE APP / NETWORK III

Now we make example 8 a real app by connecting it to real storage.

Examples 8 and 9 are similar to make a point: the storage destination for the KVS can be switched out by parameters. There is not a change in the source code needed for that.

Example 9 also demonstrates how the async calls of example 8 look when written synchronously.

- node.js
- local netork
- synchronous

examples/example9.js

```
...
var kvs

global.onPotInitialized = () => {
    bee = process.argv[2]
    batch = process.argv[3]
    kvs = pot.newSync(bee, batch)
}

function put(key, value) {
    kvs.putSync(key, value)
    return `put ${key}: ${value}`
}

function get(search) {
    value = kvs.getSync(search)
    return `get ${search}: ${value}

           <script>
           document.getElementsByName('result')[0].value = '${value}'
           </script>`
}

const server = http.createServer((request, response) => {

    var log  = ''
    var body = ''

    request.on('data', (data) => body += data)
           .on('end', () => {
                  const { button, key, value, search } = qs.parse(body)
                  switch(button) {
                  case 'put': log = put(key, value) ; break
                  case 'get': log = get(search)
                  }
                  response.writeHead(200)
                  response.end(form + log)
    })
})
.listen(3000, '127.0.0.1')

console.log('srv: listening at http://127.0.0.1:3000')
```

examples/examples9.js

RUNNING IT

This example uses the setup of example 5 – the local network, the postage batch etc. Set up example 5 to prepare for this. Then do

```
make example9
```

WHAT YOU SHOULD SEE

BROWSER

Like example 8.

TERMINAL

Basically the same as for example 8.

```
% make example9
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
/Library/Developer/CommandLineTools/usr/bin/make locnet_start
⬡ starting five local swarm nodes
on failure despite docker running, try erasing existing fdp-play containers
fdp-play start --detach
All containers are up and running
/Library/Developer/CommandLineTools/usr/bin/make .batch_id
make[1]: `.batch_id' is up to date.
open http://127.0.0.1:3000
node examples/example9.js "http://127.0.0.1:1633" "$(cat .batch_id)"

     POT JS Example 9: Node.js, local Swarm network, synchronous

     This is a self-contained web app, serving a page to receive input,
     returning the results.

     Functionally identical to example 8, just: started with different
     arguments – a network url and a batch id – by the calling make rule;
     and POT calls used are synchronously blocking instead of promises.

     open http://localhost:3000

srv: listening at http://127.0.0.1:3000
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › using bee url : ‹http://127.0.0.1:1633›
pot:  › using batch id: ‹37004b73068c18e024d8221ff55b6584…›
pot:  » created new hybrid swarm network loader
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  foo: bar
pot:  › ⟶  666f6f0000000000000000000000000…: 03626172
pot:  » get  foo: bar
pot:  › ⟵  666f6f0000000000000000000000000…: 03626172
```

BUILDING

There is no need to build POT JS to use it.

The main executable, the Go WASM file pot.wasm, the Javascript & HTML files are portable.

To build, run

```
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
```

or use

```
make build
```

WHAT YOU SHOULD SEE

```
% make build
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
```

CLEAN BUILD

To rebuild from scratch, including integrity hashes and rebuilding **go.mod**, use:

```
% make clean build
```

WHAT YOU SHOULD SEE

```
% make clean build
⬡ clean to rebuild from scratch
rm -f go.mod
rm -f lib/pot.wasm
rm -f lib/wasm_exec.js
rm -f lib/potjs.js.sha384 lib/wasm_exec.js.sha384
rm -f .batch_creation .batch_id
go mod init potjs
go: creating new go.mod: module potjs
go: to add module requirements and sums:
    go mod tidy
go mod edit -replace github.com/ethersphere/proximity-order-
trie=github.com/ethersphere/proximity-order-trie@v1.0.0
go get
go: added github.com/beorn7/perks v1.0.1
go: added github.com/cespare/xxhash/v2 v2.2.0
go: added github.com/ethersphere/bee/v2 v2.5.0
go: added github.com/ethersphere/proximity-order-trie v1.0.0
go: added github.com/hashicorp/errwrap v1.0.0
go: added github.com/hashicorp/go-multierror v1.1.1
go: added github.com/prometheus/client_golang v1.18.0
go: added github.com/prometheus/client_model v0.6.0
go: added github.com/prometheus/common v0.47.0
go: added github.com/prometheus/procfs v0.12.0
go: added golang.org/x/crypto v0.23.0
go: added golang.org/x/sys v0.20.0
go: added google.golang.org/protobuf v1.33.0
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" lib/
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go
nomock.go
⬢ √ pot.wasm built for production
```

REQUIREMENTS

POT JS includes the Go implementation of POT.

POT JS

Tested with:

- go 1.24.0
- node.js 22.17.1
- chrome 138.0.7204.184 arm64

probably works with earlier versions.

GO POT

- go 1.24.0

- testify 1.8.4
- swarm bee 2.5.0
- proximity-order-trie 1.0.0
- x/crypto 0.23.0

Which will require

- perks 1.0.1
- xxhash 2.2.0
- errwrap 1.0.0
- go-multierror 1.1.1
- prometheus
- x/sys 0.20.0
- protobuf 1.33.0

See `go.mod`.

TESTS

To test POT JS, serve the main directory and call test/test.html

or use make test

TYPES OF TESTS

```
make explain_tests
```

lists the available tests:

```
Tests can run in 2x3x2 modes:

browser / node | in-memory / simulated / network | standard / stress.

browser    web      |  Javascript running in the browser
node       node     |  Javascript running in the terminal using node.js
-------------------+-------------------------------------------------
in-memory  inmem    |  non-persistent, in-memory storage
simulated  sim      |  simulated storage for testing exceptions
network    locnet   |  local Swarm network storage
-------------------+-------------------------------------------------
standard   test     |  153 test suites of various flavors
           quick    |  like *_locnet_test but re-using the batch id
stress     stress   |  4 longer-running suites; mass & concurrent access

The following are the make rules (for other rules use % make help):

test                |  explain test modes and run node_inmem_test
webtest             |  explain test modes and run web_inmem_test
web_inmem_test      |  test api interaction with go pot in-memory persisting, web
web_inmem_stress    |  stress test with go pot in-memory persisting, web
web_sim_test        |  extended exceptions tests w/out go pot connection, web
web_locnet_test     |  standard tests with a locally installed Swarm network, web
web_locnet_quick    |  like web_locnet_test but re-using the last batch id, web
web_locnet_stress   |  stress test with a locally installed Swarm network, web
node_inmem_test     |  test api interaction with go pot in-memory persisting, node
node_inmem_stress   |  stress test with go pot in-memory persisting, node
node_sim_test       |  extended exceptions tests w/out go pot connection, node
node_locnet_test    |  test with a locally installed Swarm network, node
node_locnet_quick   |  like node_locnet_test but re-using the last batch id, node
node_locnet_stress  |  stress test with a locally installed Swarm network, node

All node_* tests are run on push by github CI workloads, except *_quick.
CI runs all those tests for ubuntu-latest, and all non-locnet for MacOS.
```

The standard tests run against the Go implementation of POT, like the use would be in production. The extended simulation tests, however, focus on POT JS and its WASM-based interfacing to Go POT, add cancellation and time out cases. They do not interface with the Go POT code but emulate it to arrive at clarity about where a potential error originates.

Latency emulation, cancelled promises, and shaky connectivity as well as retention error conditions will be added to emulate the reality of an unreliable network as ultimate storage layer.

Note that the simulation tests (**node_sim_test** and **web_sim_test**) first change the project configuration (**go.mod** + Go directives) and build an executable (**pot.wasm**) that will not work for production. They will maybe briefly appear to work but they sidestep Go POT Use **make unmock**, or **make test** or **make build** to restore the production target, i.e., build the real **pot.wasm**.

The extended simulation tests use a separate, dedicated, in-memory mock storage to separate them from the Go POT implementation. It is not the same as the in-memory storage of the Go POT module that is often referred to and that is used as default when **new()** is called without parameters.

The tests are made to work in the browser and the terminal, using node.js. In all cases (standard, extended simulation, local network, etc.) the tests are invoked by opening the file **test/test.html** (browser-side) or **test/node.js** (server-side). The file that is altered between the test modes is **go.mod**, plus directives to redirect/replace the real Go POT with **mock/**.

The test harness is designed to bridge the language divide and allow for the emulation of exceptions in one degree of separation (caused in Go, caught from JS). The major relevance of the test harness is to help surface any friction between JS's and Go's resource release mechanisms (JS promises and Go channels) to detect possible leaks. Go's channels are not without peril – they can deadlock – and the challenge is aggravated by their position behind the JS API. The relevant code triggering the exceptional conditions resides in **mock.go**. For its purposes, it has to be part of the **main** package rather than package **pot** in **mock/**.

RUNNING A TEST

```
npx http-server -c1 . &
open "http://127.0.0.1:8080/test/test.html?tag=in-mem"
```

Or use

```
make web_inmem_test
```

It uses `test.js` and `test.css` to display the results from a suite of 160 tests across the available **put** and **get** functions, and more.

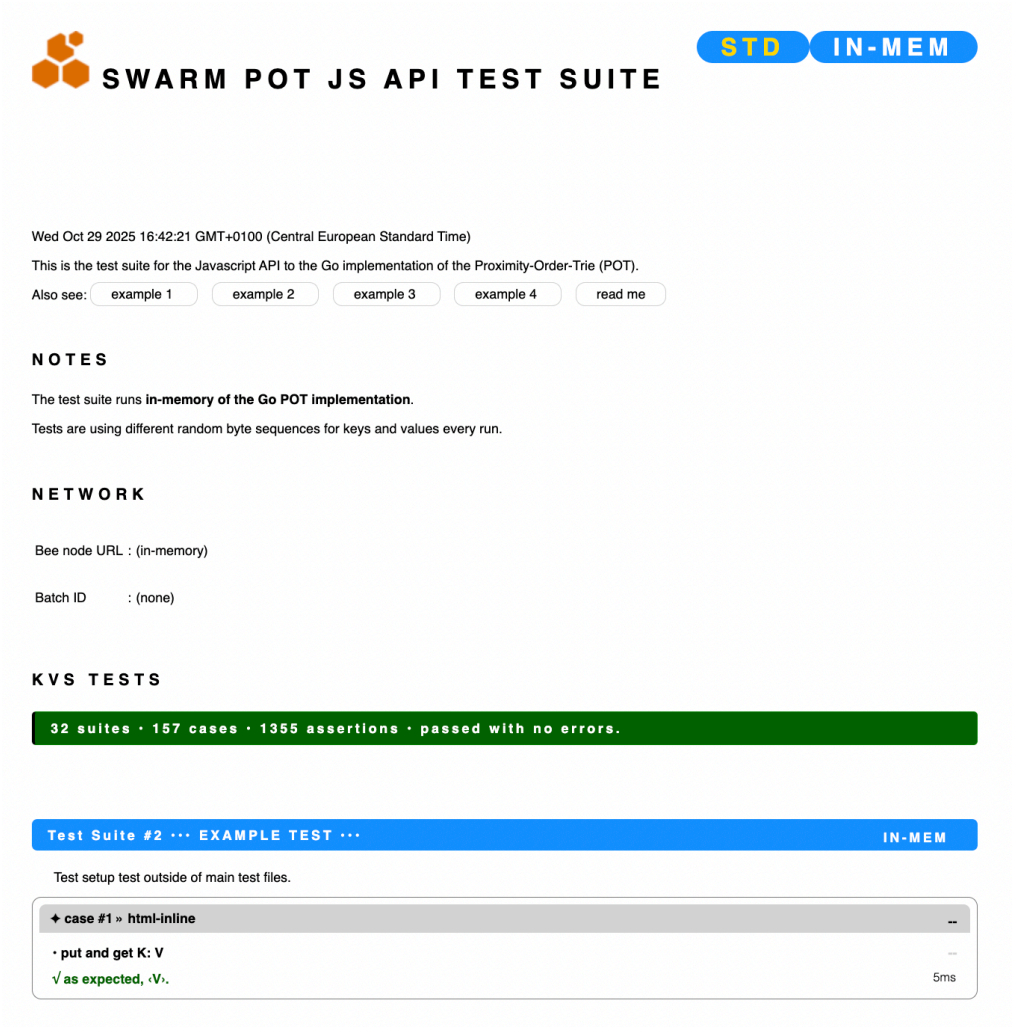
WHAT YOU SHOULD SEE

TERMINAL

If you used npx http-server -c1 . & ; open "http://127.0.0.1:8080/test/test.html?tag=in-mem":

```
% npx http-server -c1 . &
Starting up http-server, serving .

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 1 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://192.168.178.192:8080
Hit CTRL-C to stop the server

% open "http://127.0.0.1:8080/test/test.html?tag=in-mem"
...  "GET /test/test.html?tag=in-mem" ...
(node:27593) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is
deprecated
(Use `node --trace-deprecation ...` to show where the warning was created)
...  "GET /test/test.css" ...
...  "GET /lib/wasm_exec.js" ...
...  "GET /test/favicon.ico" ...
...  "GET /test/suites.js" ...
...  "GET /test/test.js" ...
...  "GET /lib/pot.wasm" ...
```

BROWSER



BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:
pot:  ••• Test Suite #2 ••• EXAMPLE TEST •••▶IN-MEM◀
pot:  Test setup test outside of main test files.
pot:
pot:  ✦ case #1 » html-inline
pot:  --
pot:  • put and get K: V
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  K: V
pot:  › ⟶  4b00000000000000000000000000000000…: 0356
pot:  » get  K: V
```

```
pot:  › ←  4b000000000000000000000000000000…: 0356
pot:  √ as expected, ‹V›.
pot:
pot:  ••• Test Suite #4 ••• SIMPLE GETS AND PUTS OF KVS, SYNCHRONOUS •••▶IN-MEM◀
pot:
pot:  --- b o o l e a n
pot:
pot:  ✦ case #2 » • put K1: false
pot:  suites.js:58
pot:  • new map
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 2
pot:  › slot ref: 2
pot:  √ no error raised.
pot:  • put K1: false
pot:  » put  K1: false
pot:  › →  4b310000000000000000000000000000…: 00
pot:  √ no error raised.
pot:  • getBooleanSync K1
pot:  » get  K1: false
pot:  › ←  4b310000000000000000000000000000…: 00
pot:  √ as expected, ‹false›.
pot:
pot:  ✦ case #3 » • put K1: true
pot:  suites.js:74
pot:  • put K1: true
pot:  » put  K1: true
pot:  › →  4b310000000000000000000000000000…: 01
pot:  √ no error raised.
pot:  • getBooleanSync K1
pot:  » get  K1: true
pot:  › ←  4b310000000000000000000000000000…: 01
pot:  √ as expected, ‹true›.
wasm_exec.js:22

...

pot:  ✦ case #157 » Oversize Value
pot:  suites.js:4576
pot:  max value size is 100000. Storing the maximal value is part of the stress test
pot:  batch
pot:  • 100000+1 byte sized value should be stopped (async)
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 121
pot:  › slot ref: 121
pot:  √ no error raised.
pot:  • put 100001 byte buffer raw B
pot:  ### put error: value too large (> 100000 bytes)
pot:  √ expected error: ‹Error: ### put error: value too large (> 100000 bytes)›.
pot:
pot:  ••• Test Suite #32 ••• FAILURES •••▶IN-MEM◀
pot:  Failure tests are available in `simulation` test mode. Use `make web_sim_test`
pot:  or `make node_sim_test`
pot:
pot:  ••• tests cases run: 157 • assertions: 1355  •••
pot:  ▶ no errors ◀
```

DEVELOPING

This chapter is about contributing to or modifying POT JS itself.

Developing and testing have been done on Mac and Ubuntu, see integration tests in `.github/workflows/`.

FILES

repo temporarily at https://github/brainiac-five/potjs

```
README.MD              Subset of this manual

potjs.go               Go JS interface
nomock.go              No-op version of test functions
mock.go                Monkey patch mock of Go POT for extended testing
swarm_nodejs.go        Hybrid Go-Javascript LoadSaver for node.js on Swarm

mock/                  Replacement of Go POT for extended testing
 pot.go                mock structures
 go.mod                Go module locations
 pkg/persister         Copy of Go POT persisters to satisfy depencies
   ...

lib/
 pot.wasm              Go POT implementation compiled to WASM
 wasm_exec.js          Generic Go WASM wrapper (replace w/your Go version's)
 wasm_exec.js.sha384   SHA384 checksum for sub-component verification
 pot-node.js           JS WASM initialization and fetch code
 pot-web.js            JS WASM initialization
 pot-web.js.sha384     SHA384 checksum for sub-component verification

doc/                   POT JS documentation
 POT JS manual.pdf     quick start, instructions, tutorial, examples
 POT JS manual.odt
 POT JS function reference.pdf
 POT JS function reference.odt

examples/
 example1.html         Briefest example, in-browser, in-memory storage
 example2.html         Interactive example, input fields, integrity
 example3.html         Example 2 without pot-web.js for more control
 example4.html         Example 1 but using synchronous functions
 example5.html         Example 1 but connected to a local Swarm network
 example6.html         Demonstrating the behavior of the save reference
 example7.js           Example 1 but running in the terminal with node.js
 example8.js           Example 2 but as a single-page node.js web server
 example9.js           Example 8 but connected to a local Swarm network
 favicon.ico           Swarm logo to suppress browser errors

tutorial/              Files t0.html – t14.js for tutorial of the manual
   ...

test/
 test.html             Test start for in-browser tests
 node.js               Test start for node.js-based tests
 suites.js             Test suites
 test.js               Generic test harness
 test.css              Generic test optics
 favicon.ico           Swarm logo to suppress browser errors

go.mod                 Go module locations
go.sum                 Go module check sums

Makefile               Build scripts

favicon.ico            Swarm logo to suppress browser errors
```

MAKE

The project comes with a versatile `Makefile` to support developing.

Below are the build rules for executables and for running tests of POT JS.

Note that building and any of the following is not required for using POT JS. The relevant executable files are portable and part of the repo, found in `/lib`.

RULES

Run with `make <rule>`

build	build executables
example<n>	start server and run example <n>. n = 1-9
test	explain test modes and run node_inmem_test
webtest	explain test modes and run web_inmem_test
explain_tests	explain test modes
web_inmem_test	test api interaction with go pot in-memory persisting, web
web_inmem_stress	stress test with go pot in-memory persisting, web
web_sim_test	extended exceptions tests w/out go pot connection, web
web_locnet_test	standard tests with a locally installed Swarm network, web
web_locnet_quick	like web_locnet_test but re-using the last batch id, web
web_locnet_stress	stress test with a locally installed Swarm network, web
node_inmem_test	test api interaction with go pot in-memory persisting, node
node_inmem_stress	stress test with go pot in-memory persisting, node
node_sim_test	extended exceptions tests w/out go pot connection, node
node_locnet_test	test with a locally installed Swarm network, node
node_locnet_quick	like node_locnet_test but re-using the last batch id, node
node_locnet_stress	stress test with a locally installed Swarm network, node
clean	prepare for building from scratch
distclean	prepare for commit to repository, including build

EXAMPLES

example1	in-browser in-memory, logging to console
example2	in-browser in-memory, interactive
example3	in-browser in-memory, alternate initialization
example4	in-browser in-memory, synchronous calls
example5	in-browser local network, logging to console
example6	in-browser local network, like 5, parametrized
example7	node in-memory, logging to terminal
example8	node in-memory, interactive web app
example9	node local network, interactive web app (same as 8)

SUPPORT & DEBUGGING

mock	prepare for extended exception tests
unmock	reset for production or non-extended tests
stop	stop local http server(s) and local Swarm network
http_serve	start local http server serving current directory
http_stop	stop local http server. Ctrl-c does not
locnet_install	install the local test network
locnet_start	start the local test network on docker
locnet_batch	buy stamp batch (free)
locnet_tests	run test suites on local test network
locnet_stop	shut local test network down
locnet_logs	show the entire log of the queen node

DEVELOPER NOTES

The following is rationale, background and learnings on why things were done the way they are.

A JS API IMPLEMENTED IN GO

POT JS, while being a Javascript API to Go code, is intentionaly and necessarily programmed almost exclusively in Go. It would not work the other way around, i.e., by implementing more – or all – of its functionality in Javascript. But it is also desirable that it has no core Javascript layer.

There is a Javascript side, **pot-node.js**, and **pot-web.js**. However, only around a hundred lines of code or less of each file form part of POT JS, and they deal exclusively with the activation of the WASM mechanism, not in POT functionality. The exception is the blocking **fetch()** function in **pot-node.js** that is needed by Go POT's synchronous **persister/LoadSaver** model to work with node.js. Even this function is just one page long. The rest of **pot-web.js** and **pot-node.js** represent the generic glue code provided by Go for WASM operation, and a modified Javascript package that enables **fetch()**. But in effect, the WASM executable, all of it written in Go, and nothing written in Javascript, constitutes what is required to use POT JS in the browser. **pot-web.js** can be left out and Go's **wasm_exec.js** included instead. Node.js needs a little more support for one specific circumstance (see Node.js Swarm Persister, pg. 87, and Synchronous Architecture, pg. 89). Regardless , the actual functionality of the API is all Go.

There is no alternative to this design, which will be explained immediately. But eventually this focus on Go is an advantage. The following explains why it is necessary but also desirable to have POT JS programmed almost exclusively in Go and details how that is the cleaner solution.

◊

POT JS is really Javascript without Javascipt code: even though POT JS is written in Go, what this Go code creates (cf. filepotjs.go) are Javascript objects and functions. For all intents and purposes, POT JS, at runtime, is mostly Javascript. Just programmed in Go. The Go code creates Javascript runtime elements that constitute the actual API. That is, methods like kvs.put() are functions written in Go but executed by the Javascript runtime like native Javascript functions, methods of a KVS. Each of which are also created in Go but are almost completely native Javascript objects.[14] The Go runtime's part is to create them, but also to execute them.

[14] The difference: the Javascript garbage collector does not feel responsible for them, but the Go GC does; and if the WASM executable crashes, they 'disappear.'

There is no way to do the opposite **using syscall/js**. I.e., there is no way to create native Go objects from Javascript. **Syscall/js** is a Go package after all, it extends Go, not Javascript. There are translators that create Go code from Javscript code, at compile time. But that is not the same as creating runtime objects for one language in the other at runtime, which is needed to create an API as tight and fast as POT JS.

That **syscall/js** works in this direction, Go to Javascript, is plausible, because the calls are coming from Javascript to Go. We want a gate from Javascript to WASM Go code, not the other way around. This is why it makes sense that the connecting elements of the API are created in Go – using **syscall/js** functions – to create Javascript objects and Javascript functions. In a pedantic sense, while the code is all Go, the API actually is fully Javascript. Just created by Go code, not Javascript code. Eventually, Javascript does not 'realize' that it is operating on 'non-native' Javascript objects that are created using Go and **syscall/js**.

Apart from that, there is a reason why at least some Go code is required and the mirror image to write the entire API in Javascript would not be possible: there are two typical classes of Go objects in play, employed by Go POT, that any Go project would contain: **contexts** and **channels**. Because they cannot be transferred over to Javascript that does not know the concept of **channels** (**contexts** are special channels), they have to be managed and 'remain' on the Go side. This is where **syscall/js** explicit[XXX] incompleteness might be hurting. Maybe there could be a way to hand handles to **channels** over to Javascript. This would likely require manual handling of garbage collection on the Go side to prevent the **channels** from being discarded as unused because the Go runtime has no visibility on the Javascript side. The overhead of keeping track of **contexts** and **channels** – either by keeping the objects, or by protecting them from premature garbage collection – necessarily has to be implemented on the Go side.

Therefore, even in its slimmest form, using **syscall/js**, one would always end up having at least some Go code in the mix. The API could never be all Javascript the way that it is now (almost) all Go.

This also trumps another reservation that might be raised: that it should seem much cleaner if one could avoid adding a main package to the Go implementation of POT in order to create a Javascript API, and to instead let Go POT remain untouched, as is, with no additional Go code added just for this specific API. Convincing as this view is, it is not feasible. The way **syscall/js** and WASM work it is a given that API-specific Go code has to be added. It can't work to just wrap Go POT into WASM and be done, all the rest being in Javascript. There has to be some Go code that creates the bridge; it cannot be Javascript code.

And that Go code has to be part of the WASM. Thus only compiling Go POT to WASM won't work. One way around it would be to make the WASM act like a server, which would be slow, not the implementation of a Javascript API, but a general interface. And it would require adding code to Go POT just the same. But to be exact, Go POT is not changed, it is just made the main component of the API project POT JS.

So, much of the objects and functions the Go code creates are Javascript. Thus, much of POT JS is really Javascript, just not Javascript code. Something must be 'the bridge' and these Go-created Javascript objects are it.

◊

Now, if the question was asked from a high level, whether having virtually no Javascript code can be a good choice for the heart of a Javascript API, the answer is that the very nature of **`syscall/js`** determines all else, and it is: to offer the possibility to 'program Javascript in Go', but not vice-versa,. However, even those aspects that would appear to unquestioningly lend themselves to be implemented in a complementary Javascript layer rather than in the Go code – like, e.g., promises – turn out to only work when programmed in Go, instead of in Javascript: because transacting between Javascript and Go can deadlock the Go code when certain functionality is written in Javascript, instead of in Go. This even includes the creation of promises, a functionality really close to the heart for Javascript.

Promises being an epitome of Javascript programming, it must seem strange that POT JS creates promises in Go. However, it simply doesn't work in the way that would look more intuitive, i.e., creating a Javascript layer that defines the promises, which's executors then call into the Go WASM functionality synchronously. Below is some background that may help to explain why this is so.

In the end, the answer to why POT JS is mostly programmed in Go is that it is not a choice.

But beyond these hard facts that favour coding POT JS in Go, there are reasons, discussed now, why it can be regarded as the optimal design; the right choice – if it was not inescapable at any rate.

◊

From a high-level it is preferrable to have code only, or mostly, on one side – Javascript or Go – and to not cross the language barrier multiple times to keep things robust and avoid unknown unknowns. Keeping things mostly on one side can minimize error sources and overhead that may occur unwittingly when switching between languages, as well as the loss of original state that this entails. **`Channels`** on the Go side, **`exceptions`** for Javascript: neither can be 'taken over' to the other side, nor conceptually substituted well, which creates problems. This is why it looks preferrable, independently of the limitations that partially dictate the answer, to keep the bulk of code either on the Go or the Javascript side.

The way **`syscall/js`** works clearly predisposes the Go side.

Keeping things simple – by avoiding language switch overs – becomes more relevant for the unsettling disclaimer that comes with **`syscall/js`**, that it is "experimental and not for production."[XXX] There seem to be no complaints that it is not stable. It may be Google's typical caution. But it encourages only more to keep things simple and to avoid more complex constructs where ever possible. Keeping it all on the Go side then, after some code on the Go side is unavoidable in the first place, appears to be the right choice to increase robustness.

But even if all that wasn't acceptable as convincing argument for pushing everything into Go, it turns out there is a – negative – technical 'killer' argument: as mentioned, it does not work to have asynchronicity being taken care of on the Javascript side. It does not work to have the promise-wrapping for asynchronous calls be coded in Javascript. Specifically, what does not work is to wrap synchronous calls to Go WASM in Javascript promise executors on the Javascript side. And when using WASM in the browser, a synchronous **`fetch`** call locks up – as the **`syscall/js`** documentation warns[XXX] – It does therefore not work to create the promises in Javascript for the purpose of a

Javascript-side layer. Promises have to be implemented in Go, oddly – as POT JS is doing. Using 'the same' logic creating the Javascript promises on the Go side, works.[15]

The `syscall/js` documentation is not clear that this would be required to avoid deadlocks. There is a description on how deadlocks should be avoided by using go routines.[XXX] But it is not explained why creating the asynchronicity on the Javascript side should be less effective. Under certain conditions, this deadlock happens every time. Creating the promise on the Go side entails a go routine to uncouple the promise executor from the synchronous program flow.[XXX] Just as the `syscall/js` documentation seems to propose. The Javascript asynchronicity inherent in the promise creation, empirically, does not suffice.

This might be because in a 'go executor' call the go routine call's mechanics are truly multithread, implemented such that if the operating system allows, go routines might run on parallel threads. Maybe this is what is preventing the mechanism from locking up (or, the go runtime to 'believe' that it is locking up) by virtue of a stronger decoupling, effected by the go routine call. In that interpretation, the fact that doing the same on the Javascript side – wrapping sync API calls into Javascript promises – always locks up, could be explained by the fact that in Javascript nothing is ever truly parallel, as a basic strategy to keep Javascript programmers safe even while the paradigm is hyper async (except Web Workers, which add true mutlithreading as their reason for existence). Go routines though are designed to run in parallel if the OS and hardware allows for it. This thesis can be seen as weakened by the fact that Go in WASM always runs on only one thread. But if the deadlock detection's false positives are the actual problem, this might make no difference.

Regardless, calling from Javascript into Go and then Go calling back to Javascript by making a fetch call seems to only ever work by go routine decoupling. Always building such a go routine into a Javascript promise makes for a desirable architecture though that has no obvious drawbacks. It even has the priceless advantage that such Go-native promises provide the only way to trigger native Javascript exceptions from Go.

As the promises are the 'most outside' hull of calls and seeing that even they cannot be implemented on the Javascript side, then little else can. But as noted, for the sake of robustness it looks desirable to have as much code as possible on one side, and to avoid crossing language lines within one call. The necessary design thus also looks desirable, even if it is surprising that eventually there is no real Javascript layer in POT JS.

This is why POT JS is implemented almost entirely in Go, and why this is a net positive.

NOTES ON DEBUGGING

Being wrapped in WASM to interface with Javascript, the Go executable is opaque. It cannot be debugged as there are no debuggers for WASM, and it becomes unavailable once it has panicked.

[15] https://github.com/brainiac-five/potJavascript/blob/86fc8af3ea6b0f148bed1efb34d8dd00190bd1b3/potJavascript.go#L50

Consequently, the development process benefits from being test-driven to unearth unexpected problems. There are 4,500 lines of test code in the `test/suites.js` file. The tests cover a wide array of cases. They include examples of failures and can be run as against a local Swarm test network, the in-memory storage, or in a special simulation mode that triggers failures.

If unexplainable errors are encountered, increasing the log level to `DEBUG` will be a helpful measure to double check assumptions. Additional logging can be added to the source code using `log(TRACE, "...")`, while setting the log level to `TRACE`.

NOTE ON SYSCALL/JS

The connection between Go and Javascript through WASM is based on Go's `syscall/js` package. This package is officially labeled as experimental by its provider, Google:

> "This package is EXPERIMENTAL. Its current scope is only to allow tests to run, but not yet to provide a comprehensive API for users. It is exempt from the Go compatibility promise."
>
> https://pkg.go.dev/syscall

This reads like a stark warning going beyond Google's typical caution. It explains the incompleteness of syscall/js, e.g., the lack of an option to directly throw exceptions. What remains unclear, and is worrying, is why this imperfect state has been going on for so long.

The disclaimer makes it just more likely that the browser WASM crashes that report deadlocks are not in fact always deadlocks but could be sometimes or always be a misfiring, overzealous deadlock detection.

JAVASCRIPT REWRITE

If Go POT were to be rewritten in Javascript, the central questions would be **a)** whether Javascript can do the bit crunching fast enough that is required to update a POT. Go WASM will be hard to beat. But if this could be shown, also for large tries, or that the system never gets CPU-bound but always stays network-bound in real operation, then the implementation of layers should where possible not follow Go POT, but be simplified, and on this path **b)** inspected whether it would be

possible to replace the persister loop,[16] which is currently implemented by recursion and requires blocking network access, with concurrent, asynchronous writes. While there are strong arguments made against the feasibility of that, it would not seem to be inherent to the POT algorithm that it would have to be implemented with blocking network access, but likely rather a price of the flexibility and multi-layered design of Go POT that would not necessarily be required for a dedicated Javascript rewrite with a less generalist focus. If it could be done, this would make the operation for node.js much more robust than the current compromise that entails a low size limit for values in an overall hacky approach that could not more clearly go against the grain of node.js – the grain being, simply, that writes are to be asynchronous. In passing this could also enable a massive performance boost. If the system turns out to be network-bound, which is likely, the parallelization of writing state to storage could be magnitudes more effective than any other fine-tuning effort. It could more than make up for Javascript's lower bit-processing speed.

INITIALIZATION

When starting out, the Go WASM executable has to be loaded, both for a HTML5 application or when using POT JS with node. This is handled by **pot-web.js**, or **pot-node.js** respectively, which also provide the wait function **pot.ready()**.

If **potjs-*.js** is used, it creates the global **pot** object[17] before **pot.wasm** is done loading and starting up and provides through **pot.ready()** a promise that can be used to wait for **pot.wasm** to complete initialization. But **pot** and **pot.ready()** are the only elements of the API that already exist at that point, no other function does until the promise of **pot.ready()** resolves.

```
await pot.ready()
kvs = await pot.new()
...
```

cf. XXX

The core of **pot-*.js** are only a few lines of code that can alternatively be programmed directly in your source code, omitting **pot-*.js**, as shown in **example3.html**. In some situations that might give helpfully more fine-grained access to design the start-up sequence. (The example happens to use newSync() without any deeper intention than to show it in action. In order to use **await pot.new()**, **.then((r) =>...** has to become **.then(async (e)(r) => ...**):

[16] Link XXX

[17] A stub, to be precise and in a different way.

```
<script src="lib/wasm_exec.js"></script>
<script>
    const go = new Go()
    var map;
    WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
            .then((r)=>{
                    go.run(r.instance)
                    map = pot.newSync()
            })
    ...
```

part of examples/example3.html

With this setup, the global **pot** object is created only after the **pot.wasm** executable has loaded and is running and only then do the POT JS functions become operational. Before that point, the **pot** object and the functions do not exist and trying to use them will cause the expectable JS exceptions to be thrown. **pot.ready()** will not exist at any point, and **onPotInitialized()** must be used instead to catch the moment that working with the **pot** function can begin. **onPotInitialized(),** if it exists, will be called by **pot.wasm** as soon as the initialization is done and **pot** exists.

```
function async onPotInitialized() {
            map = await pot.new()
            ...
```

xxx

The **pot.wasm** executable loads quickly. Most of the time, it will not be strictly needed to use **pot.ready()** or **onPotInitialized()**, as demonstrated by **example2.html**. In that example, a user has to input data in form fields, before the first operation on a **pot** object is due. This will hardly ever be fast enough to race the initialization of **pot.wasm**. The interaction coming from the user will practically always happen only after **pot.wasm** has long been loaded. It could be a source of errors though in the case of a network hic-up. So even if during development is never a problem, the start up sequence should be implemented cleanly to avoid occasional crashes for users with a shaky internet connection.

Processing in a node.js program will generally be too fast to ignore the need to wait for **pot.wasm** to finish loading. You will immediately see errors if you try without **pot.ready()** and **onPotInitialized()**. Note also that with node.js you can't leave **pot-node.js** out if you want to store to a network because **pot-node.js** contains a special implementation of a synchronous fetch that is required for the POT algorithm as implemented in Go POT and – as opposed to the browsers, does not exist anymore in node.js.

TYPES

AUTOMATIC CONVERSION

A Javascript developer will make assumptions about how types work. Javascript being weak-typed rather heightens expectations of automation of type handling and on-the-fly casting. To provide the least surprising solution,[18] Javascript types are automatically converted by POT JS to and back from the raw binary storage format values are stored in with the POT algorithm to come back as-stored. This automatism can be shirked using the `*Raw*()` functions that write and read the pure binary payload, in Javascript `Uint8Array`s.

The Go POT implementation, per se, stores raw binary data. In Javascript the equivalent would be `Uint8Array`. But that is not idiomatic to normal Javascript use. `Uint8Array`s cannot be used for keys in Javascript,[19] and cannot be compared with each other as could be expected. They are generally less practical as values than primitives for a lot of use cases in Javascript. With a tool like POT JS the expectation would be that sending a variable of a certain type to a key-value store should later get you back that variable in its type, rather than some raw data that has to be type-cast to be what it once was. To oblige the retrieving code to have the knowledge about the required type, to correctly interpret the data, can be seen as requiring a conflation of concerns.

To achieve type-awareness, type encoding has been implemented that can put and get Javascript values without losing their types. It writes a first byte to any `put*()` (that does not have `Raw` in its name) that reflects the Javascript type. Thus, `get()` or `getSync()` functions can return the correct type for the stored value. But this makes data stored with `put()` mostly incompatible with all other (raw) `get` functions. The assumption is that especially more advanced applications will prefer full control and use raw getting and setting, where programmers new to Swarm POT JS should be welcomed with a more comfortable experience, the automatic type conversion. Both approaches probably have value but the potential for errors is nasty. This is why a flag is in the works to force a user to state explicitly on creation, whether raw access should be allowable.

NUMBERS

Javascript numbers are stored to Swarm as IEEE 754 floating point standard byte code.

The reason is that the universal number type in Javascript, even integers (other than the special case of `BigNum`s), are internally stored as 64-bit floats. Direct access to the bits representing these numbers guarantees lossless conversion, especially in edge cases.[20]

---

[18] *en.wikipedia.org/wiki/Principle_of_least_astonishment – "In user interface design and software design, [1] the **principle of least astonishment** (POLA), also known as principle of least surprise, [a] proposes that a component of a system should behave in a way that most users will expect it to behave, and therefore not astonish or surprise users."*

[19] Because they do not have a natural order in Javascript.

[20] The alternative of storing numbers as strings works well, too, but goes through a much more involved process of conversion that can be assumed to be much more time and memory-consuming. While in the light of this

The principally desirable representation of *integers* by their hexadecimal byte corollary was renounced as it would have meant special casing the number conversion, i.e., handling integers and floats differently depending on their face value not by any distinction Javascript itself knows or cares about. It would make some Javascript numbers into something they are not (integers), for little tangible advantage other than the hypothetical portability of POT storage between clients of different languages.

If cross-system portability of POT data access becomes a goal, then the higher robustness of an integer representation for integers would be convincing – *if* the input could reliably be known to have been meant as an integer in the first place, which in Javascript could be difficult to ascertain. Instead, for hypothetical integer portability, an additional type marker could be added that identifies integers and allows Javascript to read them but not write them.

BIG NUMBERS

Javascript Big Numbers could not be implemented at this point because the Go `syscall/js` package – being officially incomplete, see above – can currently not handle them. It seems likely that `syscall/js` could be extended to make `BigNums` work if desired. There are receipts for it online.

While modding the standard Go package for WASM communication with Javascript opens a can of worms, it seems important enough eventually to allow for `BigNums` to be stored in Swarm natively.

KEYS

As explained above in the Instructions (pg. **Error! Bookmark not defined.**), there are multiple ways that the same key can be expressed. It may be desirable to guarantee that no two keys ever access the same value.

The current situation exists because there are multiple types allowed for a key, which is idiomatic to Javascript. Javascript `numbers` are converted to their `float` byte representation and zero-padded by POT JS to arrive at the binary 32-byte key that the POT algorithm requires. `"1"`, `1`, and `"0x01"` will thus be different keys but `1`, `1.0`, `0x1` and `01` the same. The 8-byte float representation might itself be matched by a string – like the string `"A"` matches the bit representation of `131,072` in 64-bit IEEE 754, `(0x4100...)` And the keys are padded with zeros, which presents an unescapable problem for `Uint8Arrays`, unless the only allowable key was any 32-byte `Uint8Array`.

Changing this would necessarily require to make `Uint8Arrays` the only allowable keys, or to drop `Uint8array` as allowable input for keys, or to implement type-coding for keys like it exists for values.

---

context this does not matter too much, it might also be more prone to errors in edge cases such as the likes of `10/3`, `-0`, and *multiple* variations of `NaN` in IEEE 754.

Because even if the 8-byte string/number match would be ignored as rare fluke (which would not be good) any encoding of `number` or `string` keys could also be expressed with a `Uint8Array` in what would by Javascript standards not intuitively be regarded by the same value. On the other hand, being somewhat ad-hoc with automatic type casting is a fit with Javascript style.

TESTS

To accommodate for the dual origination of errors – on the Javascript or Go side – the combined test suite utilizes the same JS ↔ WASM ↔ Go channel that the API itself uses to trigger, evaluate and log the tests, jostling between languages. A number of functions were added to enable black box testing. They are no ops in production, neutered via Go build directives.

To be able to test its reaction to delays, stalls, exceptions and cancellations, and to insulate tests of the JS API from any potential problems with the Go POT implementation itself, which is not yet battle-hardened or extensively tested, the 'extended simulation' tests implement their own mock back end (`mock.go`). This is a second in-memory storage variant, separate from the in-memory storage the Go POT implementation itself offers that is used in the 'standard' tests.

This mock back end, besides being helpful for debugging, exists to emulate fringe conditions that could not otherwise be tested but must be expected when the Go POT package operates connected to a public network. E.g., mock delays are essential to test the basic user cancellation and time out design that has to satisfy both Javascript and Go language idiosyncrasies to prevent resource leaks in Go from dangling sub routines that were not freed when a request did not succeed. As a special complication, both runtimes have their own garbage collector, which always add opaqueness but, in the instance, have to be made to work together in their estimation of what resource can be discarded. In this regard, Go channels are a special challenge, as well as a potential source of deadlocks. This is why the 'simulation' creates an environment that tests the API alone, with harsher conditions.

Regarding stress tests it is unclear whether the existing stress tests put maximal pressure on POT JS and by extension, Go POT, in terms of concurrent operation.

ERROR HANDLING

The divergent error handling philosophies of Javascript and Go had to be balanced. A Javascript programmer does not expect errors coming back as return values – like in Go – but as exceptions. But Go cannot trigger Javascript exceptions through WASM. It can `return` a value of type `Error` as JS-native `Error` type return value but not `throw` it directly, due to the incompleteness it would seem of `syscall/js` – or it may be impossible to implement.

While Javascript **throw** is not available in Go, a middle ground is provided by JS **promises** that enable – when created in Go – by the use of their **reject** function a means to trigger an exception in Javascript. This constitutes an additional reason for why the API functions that return promises are preferrable. It allows for the most seamless transportation of errors from inside Go to Javascript in the expected way for a JS programmer. And calls from JS to WASM, the internet finds, should generally be promises as the nature of WASM is not quite like a library without agency but more like a parallel process. (It is not as it is executed on the same thread as the JS main thread, but it has to keep running to be available. Its main is put in a never-ending wait with **<-c**).

Synchronous POT JS calls (**putSync()** etc) cannot benefit from this way of throwing errors. They are made to return a Javascript **Error** object instead of the expected return value. This does not fit well in the Javascript workflow but there seems to be no better solution. It does not work to create a **promise** on the Go side that in case of an error immediately rejects. This still leads to asynchronicity, literally, between the error and the arrival of the error message, and **"uncaught (in promise)"** type errors that arrive a little bit late. E.g., often after a test program has finished.

A second solution is described in
https://stackoverflow.com/questions/67437284/how-to-throw-js-error-from-go-web-assembly:
the WASM compiler can be extended using a hint, and the generic **wasm_exec.js** enhanced to add the possibility to **throw** from Go.

Because this seems only interesting for synchronous functions, which are not assumed central to the overall user experience, the effort and the risk of touching the compiler and **wasm_exec.js** – where until now they are used without modification – seem inappropriate. The less-than-optimal solution will remain that synchronous functions return error objects when things go wrong, and that async functions that return promises, and throw errors, should be preferred.

SLOTS

The array in **potjs.go** of **Slot** structures, **slots**, keeps the part of the state of a KVS on the Go side that cannot be transferred to Javascript. A different, more elegant solution would be preferrable. As things are, given the limitations of **syscall/js** (see pg. 77) this might be the best solution. In what is a bit of a mine field, they are sufficiently simple that they can be made robust.

The slots hold the Go objects and **channels** for which how to create a safe Javascript handle for is not obvious and would likely create garbage collection issues. A handle for a Go object that could, e.g., be created from its memory address and given to Javasscript, might have the effect that Go destroys the object if it detects that it has become unreachable by Go's count. Go's **runtime.keepAlive()** is designed to work within one function. Aa strategically created closure could keep variables alive within it and could prevent the collection. But for its higher complexity it would create no less overhead than a straight array of state that holds the information and in passing prevents its collection.

A yet more involved solution could be tried by enclosing Go variables in a `js.Func` closure that can then be accessed through a wrapping Go function because the variable exists in both the inner and outer closure. But the outer closure itself cannot be attached to the JS object itself and would still have to be held somehow on the Go side – e.g., in an array. One would again resort to a map or an array to retain access to the relevant Go objects. This looks no better than storing the state itself in a map or array.

A thinkable alternative to the `slots` array would entail to use closures to hold the state itself thst cannot be transferred to Javascript and attach those closures to the Javascript object that is created for a new KVS. This works, relatively elegantly, for the Go context `cancel()` functions of the JS promises.[21] Such Go function could be wrapped in a `js.Func` – a Javascript function that is created in Go and visible on both sides – and could then be called by calling the wrapping JS functions from within Go (instead of from JS). State could be accessed from Go when wrapped in this way despite the fact that a `js.Func` must normally be a Javascript object. However, this is expressed by the `js.Func` type returning the most malleable Go type, `interface{}`. This can be used, even conditionally (i.e., under some circumstances such function could also return a `js.Value`), to return a `channel` out of the function that is then cast from `interface{}` to `chan` by the receiver. This was tried half-way and could work but again looks no more elegant than keeping the bland `slots` array.

This is why a state array like `slots` may be the best solution, even while the `cancel()` functions of the promises work without such a central list. It simply keeps Go state on the Go side.

The index of a `Slot` in the `slots` array + 1 is used as the handle that is given to the Javascript KVS object as a Javascript number: `this.slot_ref` on the Javascript side holds it. A number can be passed to Javascript without a problem – even if there is the theoretical caveat that by necessity the integer is converted to a float, as JS does not have native integers, except for `BigNums` that `syscall/js` cannot handle. Thus, theoretically a conversion to a string would be safer. But in practice, with Javascript using 64-bit IEEE 754 floats, the first integer conversion to fail is 9,007,199,254,740,993. For an application creating one KVS on average per second, this will fail at about the time when all landmass on Earth has merged into one supercontinent.

However, for security reasons, the integer count should be replaced by hashes at some point and the Slots array into a map.

The `Slot` struct is retrieved on every call to `get()`, `put()` etc. accessed through the `this` pointer that is the first parameter that every `js.Func`-wrapped Go function receives when it is called from JS. `this.slot_ref` is used as index `+ 1` to access the right Go `Slot` struct of the KVS that is being operated on from the slots array. The +1 keeps 0 an invalid `slot_ref` as a safety measure. The `Slot` struct has a double link that is compared to the `slot_ref` value to make sure that the right structure is being accessed.

The most important element in the `Slot` struct is the Go `SwarmKvs` pointer to the struct that is returned from `newSwamKvs()`. It holds, via Go `pot.SwarmKvs`, `pot.Index` that is the struct

[21] Link XXX

holding the channels that are required by `muxProcess()`, which is the function that loops through a `select` that effects a write lock, sequencing write accesses to a POT. Every KVS has its own. It kicks in when it is created with `pot.new()`.

ALTERNATIVES

In the face of the affordance to store the channels, it could be a worthwhile investigation to ask if the channels, which serve exclusively as a write-locking/sequencing mechanism, are necessary in the first place. Especially given that the entire system is strictly single-threaded and in part suffering from this. It is possible that muxProcess() – for this use case, with Javascript – is overkill.

Possibly, a different locking mechanism than `muxProcess()`'s `select` could be implemented on the Go side as Javascript's single-threaded nature could be taken to represent a free replacement for a multiplexer. Hypothetically, `muxProcess()` was not needed in its current form when interfacing to Go POT from Javascript, neither from the browser nor from node.js, unless Worker threads were employed that would bring with them true multi-threading. The logic behind the question would be that Javascript is generally single-threaded and thus needs no protection against parallel writing. It does for this reason not have primitives against it like Go has in its channels. One might even ask whether the locking aspect is a duplication of sequencing. Because JS, being single-threaded by nature, does nothing in parallel ever, which `muxProcess()` doubly makes sure can't happen (for writes), which might unintentionally cause the not quite explained deadlocks of sync calls in the browser. Perhaps the channels' and `muxProcess()`'s constantly waiting `select` might not be required in the first place, and thus, the channels could possibly be left out altogether or could be global, once per thread and not per KVS, and/or created per-call instead so that they do not have to be kept available between calls – in the `slots` array – with the KVS object. This could be a modification to make Javascript and Go play together more nicely.

If this would work, it would leave the second type of `channels` to deal with, the `contexts`. They are presently created per-call so they do not have to be kept around between calls, at the cost of losing the possibility of a KVS-wide cancellation option. A `context` for an entire KVS that all per-call `contexts` derive from would also have to be held somewhere. However, the relevant element of such `contexts` is the `cancel` function and that can be kept around in a `js.Func` closure attached to the KVS.

The last relevant object held in Slots then would be the `persister`. As this is not currently keeping a connection alive it could in its production variant – that connects to a Bee node via `http` – be implemented in a form where its state could be transferred to Javascript, if it needs state at all. Alternatively, the `persister` could be architected to be a native Javascript object, implemented in JS (code). The bigger obstacle in this case would be the special, state-holding, in-memory version of the `persister` that has to be kept alive between incoming calls from JS to not lose its state. But the overall architecture should not be bent to make this less-important, in-memory variant of the `persister` easy to implement. To the degree it is practical for testing, it should rather be the `persister` code itself where the kludge should be residing that makes state-holding between calls possible; instead of burdening the overall architecture for it.

But even for such streamlined version of Go POT and POT JS where Go POT is altered to work better with POT JS, `muxProcess()` might turn out to not be superfluous:

It means banking on that there will never be two write calls overlapping each other coming from JS in a way that would trip up Go POT. Apart from the fact that some future browser optimization might break guarantees that are currently implicit but not spelled out. A browser might do something 'helpful' that ends up using more than one thread to speed JS up and by that break the single-thread assumptions. This is not unrealistic because any measures such improvement would require building into the JS runtime would be invisible and unavailable when control is given over to WASM.

But even as is, JS' asynchronicity could be enough to have two write promise's executors end up switching back and forth in a way that they look parallel to the Go side. In the end the promises are there to make things look like they happen in parallel and have multiple things pending to speed things up. That's exactly what `muxProcess()` is there to prevent for writes. Multiple writes can pend but they must not happen at the same time, and this sequencing is what `muxProcess()` provides. JS would push things up to the point where it looks like the writes are both happening and then wait for the result, it does not nicely queue them. This would mean that `muxProcess()` is required, even for a single-thread system.

Finally though, Go compiled to WASM does itself only run on one thread and cannot use its potential to multithread. It is not the case that a speed up through parallelizing and multiplexing could be expected, with parallel writes for different KVS, even while parallel writes for a single KVS is what `muxProcess()` exists to prevent. It is also not in this constellation the case that Go's potential for multi-threading is a useful capacity of Go's that Javascript just can't deal with and that it thus would have some high-level consistency that one had to park the controlling state (the `channels` `muxProcess()` uses for locking) of this Go superpower outside. Go itself can when compiled to WASM not do what `muxProcess()` is designed to prevent it from doing in the wrong moment: it cannot work things in parallel. It would be plausible to expect that `muxProcess()` will at least prevent that two writes race each other by switching back and forth between the two greenlets in an unlucky way. However, to achieve the same protection, it would be enough to just not use go routines inbetween to make sure that no two writes could race each other because WASM is executed on a single thread by Javascript, without switching during execution. The conclusion would have to be that `muxProcess()` is unneccesary when Go POT is used in WASM. But this is a special case for Go POT, unlike its potential use in different environments, e.g., when used in Go as a library or package of a bigger system, e.g., included in a Bee client. Only when used with Javascript there is no need to take care of the case that `put()` calls could be coming in on different green threads, much less truly parallelly. So a change or re-implementation of Go POT would have to be only for this case, or other cases where Go POT was used in the form of WASM. But that would have to be weighed against the approach to re-implement the POT algorithm in Javascript and do away with the entire dual-language overhead.

Together with surprises and limitations that surfaced while joining Javascript and Go, it looks difficult to judge whether an attempt at improving this situation would be worth the try. Certainly, not at this point.

SLOTS MIRROR A POWER IMBALANCE

In the end, in a holistic view, the core of the problem is, in the first place, that the sequencer state – the `muxProcess()` channels – would ideally be carried over from Go to Javascript to be held there, which we can't and thus have the `slots` array for; while what's technically only needed, instead, would

be to either make sure there is no asynchronicity started within Go POT during the handling of one `put()`, or, to have a simple semaphore per-KVS on the Javascript side that makes sure that no two write calls are triggered from JS at the same time.

In the latter case, there could still be multiple promises for `puts` created and pending at about the same time, but they would simply have to prevented from running their executors to the blocking point of a write at the same time, and so to not rely on asynchronicity alone that would only avoid blocking for Javascript but not a write race on the Go side. Abstractly, this would seem to not change much, because at present, that queuing is happening in `muxProcess()` on the Go side. But it would be a simpler model if that was handled outside of the Go POT core, more robust and possibly apt to avoid the deadlock crashes[22] of the Go WASM executable that might be false positive of an overly sharp deadlock detector of the Go runtime (that was historically too lenient).

However, overall, making Go POT cater to JS's needs would be an optimization the time of which has not come. And in light of all the considerations above, the simplistic `slots` array looks like the best solution at least at this point. Its inelegant ways are somewhat explained by how it is an artefact of the overall architecture not being optimal from the Javascript perspective. With its possibility for true parallel execution, Go POT is more powerful than what JS can make good use of. Figuratively speaking, that power needs to be parked somewhere, unused, in the form of the `channels`.

Go POT on the other hand is meant to have the capability to work truly in parallel. Perhaps, on the one hand, the index updates could with big tries tilt from network-bound into processing-bound. That is, the bulk of the wait-time could at some point be caused by the calculations happening (in Go POT, i.e., in WASM) rather than being caused by the wait for an answer back from the Bee API. On the other hand, how `muxProcess()` is crafted is just the go way of implementing asynchronous calls to the storage network.

In this view, the construct of the `slots` array is an artifact that betrays a friction across the overall implementation – joining Go and JS – and stemming necessarily from that Go POT was not made as a Javascript backend, of course, and can do more than Javascript can take from it, namely work in a truly parallel fashion. The `slots` array deals in exactly those Go elements that JS cannot hold, i.e., the channels that serve something that Javascript was not designed to do, real multi-threading that needs real locks. And as a consistent consequence, Javascript does not have the means to hold the special type of state – the channels – that is native to Go to make true multithreading possible.

NODE.JS SWARM PERSISTER

The topic of network access for WASM executables run by a node.js script is relatively sparsely discussed on the internet and some information found there is misleading. There is a simple answer:

[22] To be clear: that happen preditable and reliably, not

NO NETWORK ACCESS FOR WASM UNDER NODE.JS

Different from a browser, Node.js does not allow any network access for a WASM executable. DNS, TCP, UDP, any fetch access fails for Go running in WASM when executed by node.js. It works in the browser though. This limitation is partially documented but the situation is less than fully clear. In the end, WASM was made for the browser – uniquely, allow for speed like compiled languages. That WASM could also be used with node.js was, it would seems, an afterthought. WASM with node.js is implemented to a degree as 'emulation.' Node.js was also not made to host things like a WASM executable.

At any rate, while in the browser, network access is extended to the WASM executable 'for free.' The sandbox that restricts the WASM code when running in node.js is stricter. Plausibly, because there is no sandbox around node.js like there is around a *browser* itself, which separates it from the filesystem. Node.js lives on the other side of that divide, with full local access.

HYBRID PERSISTER

A new `persister`[23] in the mold of the Go Swarm `persister` had to be written for node.js to work with the Swarm network. While the original Go Swarm persister works for POT JS when operating in the browser – i.e., Javascript running in `<script>` tags on a web page – it does not work when used with node.js, i.e., for Javascript projects run locally from the command line.

The new persister, `SwarmNodeJsLoadSaver`, is written in Go and largely identical to the Go `SwarmLoadSaver`. The difference is that it delegates network access to a fetch function implemented in Javascript, `jsFetchSync()`, that uses the code of the node.js XMLHttpRequest emulation package `XMLHttpRequest-ssl`, with a modification, as described below, to realize synchronous network access for node.js, as required for the persister architecture, and otherwise near impossible to realize with node.js.

This is an essential step towards a pure Javascript persister, even while other details of it are not entirely clear yet. However, the bolted-on way that `XMLHttpRequest-ssl` is implemented, together with the fact that there seem to be no better alternatives casts a shadow over the idea of a pluggable pure Javascript persister. A browser-only version for this concept does not look like a strong decision. But any node.js version, with the idea to have persisters be supplied by users (developers), would always have to find its own creative ways to realize blocking writes and reads, or itself use the modified version of `XMLHttpRequest-ssl`.

[23] The persister is a separated-out module that realizes the most basic primitives to store and retrieve for Go POT. It can fundamentally only write data to whatever the target storage is or retrieve it from there. Go POT originally has two persisters, one that stores everything in memory and one that stores everything to a Swarm network.

SYNCHRONOUS ARCHITECTURE

The Go POT implementation is fundamentally a synchronous architecture while POT JS must be asynchronous to work for node.js.

Go POT waits for writes to succeed before continuing its tasks in its `persister`, and has a locking mechanism, `muxProcess()`, specifically to make sure that only one write can proceed at a time. It is unclear whether the algorithm of POT itself demands blocking waits for every write, especially because the network writes appear to be from cache and the immutable, content-addressed entries should be persistable in any order without causing race conditions. While it appears plausible that the progressive calculation of the hashes that tie the trie into one could require a sequential handling of its (updated) nodes, the centrality of the problem caused by the necessity of blocking writes for this, would justify a separation between storage and hash calculation if that is the 'blocker.' And since the current implementation does not have a pick-up strategy if a write at first goes wrong, a failure with concurrent writes, which would at first glance appear to always be more catastrophic, might not be more difficult to repair than a failure in the sequential iteration. However, in practice, it makes sense to assume that this cannot easily be changed and will not be changed because it would in all likeliness be asking for major re-architecting of how Go POT works and how it is compartmentalized into modules.

Conversely though, Node.js quite successfully strives to make synchronous writes all but impossible. Its `fetch` does not have a synchronous option but works only as `promise`. `XMLHttpRequest`, which still works in the browsers and can be used synchronously, has been retired for node.js. There exist npm packages to bring it back and they are so popular that one of them (`XMLHttpRequest-sll`)[24] registers 6M downloads a week. This package employs a blunt hack to evade Javascript's enforced asynchronicity: it spawns a child process that handles the network request asynchronously, then blocks, waiting for the child process to terminate, which it normally does after the response callbacks are completed. The way it is implemented includes creating the child process code as a string, with the payload data hardcoded into it, to start it in a new shell. Headers are singled out as malfunctioning in the docs. Binary payloads and responses did not work and had to be added and altered.[25] For POT JS this method (that can be improved by using a file to transfer the data to the shell process) currently means a limitation for its value size (to max. ~100,000 byte). That this seems to be the accepted way to attain blocking fetches for node.js, just demonstrates how averse node.js is to synchronous network access. But it seems to be the only viable option to create synchronous network requests for node.js that is not worse than everything else.

Of course, node.js' strategy to try to deny programmers blocking fetch calls is sound. Go POT is the outlier demanding them. It would be worth it to find out whether it really needs them.

Requiring `XMLHttpRequest-sll` as a dependency and then modifying it in-situ seemed less sensible as it could be installed in different locations and with limited access. The recent attacks on NPM are an additional factor to not want avoidable npm installs in the tool chain.

[24] Link to npm XXX

[25] The alterations are minuscule, only spanning three lines. But there is no way to make the original package do it without the modifications.

GARBAGE COLLECTION

For a long-running and/or heavy-duty server, resources should be released when not needed any longer to avoid crashes memory leaks. Because Javascript and Go both have a garbage collector and have to be kept from collecting what from their side looks like a loose end,[26] this does not happen fully automatically. In fact, the garbage collection process even has to be manually stalled on the Go side so that objects that are in use on the Javascript side are not thrown out by Go to which they look out of reach.

Both Javascript and Go have rudimentary mechanisms to free resources manually, designed to, e.g., release file handles of files that might otherwise stay open when the controlling object is collected because it fell out of reach, but had not closed its files properly.

The same way, the POT JS **`Slots`** and KVS objects have to be 'released' on the Go side, once they are not needed any more. Ideally, this process would be triggered from the Javascript side, when a KVS object goes out of reach. However, it turns out that Go-created Javascript objects (cf. pg. XXX) are never collected by the Javascript GC – no matter how high the pressure on the garbage collector is amped up – while other, normal objects of the same age are getting collected by it.

The Go garbage collector, on the other hand, likes to discard the KVS objects it created right away (see log screen) when they are not protected by writing them into the **`Maps`** (**`./potjs.go`**) list on the Go side. This means that there seems to be no automatic trigger available for collection (no matter how delayed). Javascript seems to not 'feel responsible' for the KVS objects that are created in Go and Go seems to be oblivious to whether the KVS objects are in- or out-of-reach on the Javascript side. And while the Go garbage collector can be triggered manually, this is not the case with Javascript, where only the simulation by a lot of expensive memory allocations can trigger a GC run, indirectly, and unreliably.

```
pot:
pot:  ✦ case #135 » Crossover Concurrent Putting, Getting, Saving, Loading, Switching Maps
pot:
pot:  Crossover concurrent putting, getting, saving, loading, switching maps across
pot:  threads, with promise
pot:
pot:  1: • new map
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 85
pot:
pot:  2: • second new map
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 86
pot:
pot:  • waiting for completion
pot:  » concurrent threads: 2
pot:  1: √ no error raised.                                                    3 ms
pot:
pot:  1: • putK1: V1
pot:  » put  K1: V1
pot:  2: √ no error raised.                                                    1 ms
pot:
pot:  2: • get K1 from second map
pot:  » get  K1: undefined
pot:  1: √ no error raised.                                                  « 1 ms
pot:
pot:  1: • get K1
pot:  » get  K1: V1
pot:  2: √ as expected, ‹undefined›.                                           1 ms
pot:
pot:  2: • put K1: V1 to second map
pot:  » put  K1: V1
pot:  ◉ KVS on slot 84 cleaning up
pot:  ◉ KVS on slot 83 cleaning up
pot:  ◉ KVS on slot 82 cleaning up
pot:  ◉ KVS on slot 86 cleaning up
pot:  ◉ KVS on slot 85 cleaning up
pot:  1: √ as expected, ‹V1›.                                                  4 ms
```

There are scholarly articles about garbage collection and determinism of execution in Go. And the source code of **`syscall/js`** – the package that allows to interface with Javascript – reveals that Go is controlling the Javascript values'

[26] Garbage collectors discard an object when they detect that it cannot 'be reached' by the program anymore because all handles to it went out of scope – variable names, list entries, uses as a key, closures etc. However, they do this in unpredictable frequency, mostly not the moment the object cannot be reached anymore, and in any order. The clean up callbacks that can be registered are also not necessarily triggered while the object is still accessible but possibly much later. This makes sense to give the garbage collectors all the freedom they need to optimize resource utilization an\\\\\d minimize the time they require for their operation.

collection manually. They get a finalizer set,[27] which calls into WASM. Digging deeper, one runs into unmistakably direct warnings like "Don't try to use this directly."[28]

The `Maps` list in `potjs.go` serves to keep the KVS objects, `jsMap`, alive. Without them, during a normal test (`make test`) with Go POT in-memory storage the KVS objects are collected, sometimes almost immediately as can be seen from the "`◉ KVS on slot .. cleaning up`" log messages in this test run log.

TEST HARNESS

The test harness is custom made to be able to run the same suites in both browser and node.js without changes; to be able to emulate specific kinds of concurrency spanning Javascript and node.js, and failures that can occur in the intersection of the two runtimes; and to seamlessly simulate Go POT to test POT JS alone, to be able to vouch for its correctness, in case of bugs of unclear origin.

The harness consists of `test/test.js` and `test/test.css`. The test suites are in `test/suites.js`. The tests are started via `test/test_node.js` and `test/test.html` (for the browser). They are coordinated via `Makefile` rules that are complex enough that they have to be regarded as part of the test mechanism. `make explain_tests` and `make help` list the different tests. See xxx (make rules).

The test harness, i.e., the capabilities of `test.js`, are documented by the comments in `test/test.js`. All its features are employed across the 4,500 lines of tests in test/suites.js and it is straight forward to build new tests analogue to the existing ones.

The general pattern for synchronous function tests is as follows:

```
T.head("...")

T.start("...")

T.log("...")
...
T.assertNoError(t0, T, !map)

T.log("...")
 ...
T.assertEqual(t0, T, val, val1)
 ...
```

[27] https://cs.opensource.google/go/go/+/refs/tags/go1.25.3:src/syscall/js/js.go

[28] https://pkg.go.dev/golang.org/x/mobile/bind/seq#pkg-functions

```
T.start("...")
 ...
```

For async tests:

```
T.head("...")

T.start("...")

try {
   T.log("...")
   ...
   T.assertNoError(t0, T, ...)

   T.log("...")
   ...
   T.assertEqual(t0, T, val, val1)

} catch(err) {
   T.attestUnexpectedError(t0, T, err)
}

T.start("...")

try {
...
}
```

ROADMAP

Possible future improvements:

- NPM PACKAGE
- ELEMENT DELETION
- FUNCTION TIMEOUTS
- FUNCTION CANCELLING
- JAVASCRIPT MODULE MODE
- HIGHER LIMIT OF NODE.JS VALUE SIZE
- HIGHER COVERAGE OF PARAMETER TESTS
- JS-SIDE GARBAGE COLLECTION TRIGGERS
- MEMORY LEAK TESTS
- HASHES FOR INTERNAL SLOT REFERENCES
- PROOF RETRIEVAL FUNCTION
- TREE WALK FUNCTIONS
- JS-NATIVE PERSISTERS*
- TYPESCRIPT VERSION
- GO-SIDE RESOURCE TRACKING INSTRUMENTATION
- USE OF JS NEW OPERATOR*
- ERROR MESSAGE CLEAN UP